

**MATEMATICKO-FYZIKÁLNÍ
FAKULTA**
Univerzita Karlova

DIPLOMOVÁ PRÁCE

Martin Struk

Rozbíjení symetrií pro řešení splnitelnosti

Katedra algebry

Vedoucí diplomové práce: Mgr. Mikoláš Janota, Ph.D.

Studijní program: Matematika pro informační
technologie

Praha 2026

Prohlašuji, že jsem tuto diplomovou práci vypracoval(a) samostatně a výhradně s použitím citovaných pramenů, literatury a dalších odborných zdrojů. Beru na vědomí, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., autorského zákona v platném znění, zejména skutečnost, že Univerzita Karlova má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle §60 odst. 1 autorského zákona.

V dne

Podpis autora

Chtěl bych poděkovat vedoucímu práce Mgr. Mikoláši Janotovi, Ph.D. za všechny rady, připomínky, vstřícnost, ochotu a motivaci během vedení této práce.

V rámci této práce byla také vypracována kratší verze práce v anglickém jazyce, která byla předložena a přijata na The 33rd International Workshop on Experimental Evaluation of Algorithms for Solving Problems with Combinatorial Explosion (RCRA 2026).

Název práce: Rozbívání symetrií pro řešení splnitelnosti

Autor: Martin Struk

Katedra: Katedra algebry

Vedoucí diplomové práce: Mgr. Mikoláš Janota, Ph.D., Katedra algebry

Abstrakt: Symetrie v problémech splnitelnosti booleovských formulí (SAT) často způsobují nadbytečné prohledávání ekvivalentních oblastí prohledávacího prostoru, což výrazně snižuje výkon SAT řešičů. Klausule pro rozbívání symetrií tento problém řeší eliminací symetrických řešení při zachování splnitelnosti. V této práci se zabýváme generováním těchto klauzulí ze známých symetrií pro zadanou SAT formuli. Implementujeme nástroj pro konstrukci klauzulí pro rozbívání symetrie s využitím vzorů, což jsou kompaktní reprezentace podmínek zavedené v předchozí práci. Tento přístup umožňuje generovat klauzule pomocí vybraných množin permutací a zároveň řídit hloubku generování klauzulí, čímž umožňuje volit kompromis mezi silou rozbívání symetrií a velikostí výsledné formule. Hlavní důraz je kladen na experimentální zkoumání toho, jak různé množiny permutací ovlivňují účinnost rozbívání symetrií. Experimenty jsou prováděny na problémech, které lze přirozeně reprezentovat jako binární matice, ale je možné uvažovat i obecnější problémy. Hodnotíme jak rychlost řešení, tak redukci počtu symetrických řešení pomocí moderních SAT řešičů.

Klíčová slova: booleovská splnitelnost, rozbívání symetrií, generování klauzulí, permutační grupy

Title: Symmetry Breaking Constraints for SAT Solving

Author: Martin Struk

Department: Department of algebra

Supervisor: Mgr. Mikoláš Janota, Ph.D., Department of algebra

Abstract: Symmetries in Boolean satisfiability (SAT) problems often cause redundant exploration of equivalent regions of the search space, significantly degrading solver performance. Symmetry breaking constraints (SBCs) address this issue by eliminating symmetric solutions while preserving satisfiability. In this work, we study the generation of SBCs from known symmetries for a given SAT formula. We present a tool for constructing SBCs using patterns, a compact representation introduced in previous work. The proposed approach generates clauses using selected sets of permutations while allowing control over the depth of generating constraints, providing a trade-off between the strength of symmetry breaking and the size of the resulting formula. Our main focus is an experimental evaluation of how different sets of permutations influence the effectiveness of symmetry breaking. The experiments are conducted on problems naturally representable as binary matrices, but more complex problems may be considered. We evaluate both solving performance and the reduction in the number of symmetric solutions using state-of-the-art SAT solvers.

Keywords: boolean satisfiability, symmetry breaking, constraint generation, permutation groups

Obsah

Úvod	6
1 Teoretická část	8
1.1 Základní definice	8
1.2 Direktní součin grup	11
1.3 Kódování podmínek	12
1.4 Popis problémů a jejich kódování	13
1.4.1 Problém holubníku (Pigeonhole principle)	13
1.4.2 Problém N královen (N-queens problem)	13
1.4.3 Problém nalezení matice s řádky a sloupci se zadanými součty	14
1.4.4 Problém latinských čtverců	15
1.4.5 Problém společenských golfistů	17
2 Technická dokumentace	18
2.1 Generování vstupních instancí	18
2.2 Generování klauzulí	19
2.3 Pomocné funkce	20
2.4 Generování klauzulí z porovnání dvou vektorů proměnných	21
2.5 Generování klauzulí pro rozbíjení symetrií	25
3 Experimentální část	27
3.1 Problém holubníku	28
3.2 Problém N královen	31
3.3 Problém nalezení matice s řádky a sloupci se zadanými součty	34
3.4 Problém latinských čtverců	36
3.5 Problém společenských golfistů	38
3.6 Shrnutí experimentálních výsledků	40
Závěr	42
Literatura	44
Seznam obrázků	46
A Přílohy	47
A.1 Zdrojový kód a data pro experimentální část	47

Úvod

Cílem této práce je implementovat nástroj, který pro vstupní SAT formuli vygeneruje klauzule pro rozbíjení symetrií pro zvolenou množinu permutací na proměnných, a dále zkoumat, jak různé výběry množin permutací ovlivňují celkový počet rozbitých symetrií, množství přidávaných klauzulí a celkově efektivitu nástroje.

Problém splnitelnosti booleovských formulí (SAT) je jedním ze základních rozhodovacích problémů [1]. Cílem je určit, zda existuje ohodnocení proměnných zadané booleovské formule takové, že výsledný výraz je pravdivý. Přestože se nástroje pro řešení SAT (SAT řešiče) instancí výrazně posunuly kupředu, složité instance často obsahují redundance a struktury, které stále příliš rozšiřují prohledávací prostor. Jedním ze zdrojů takové redundance jsou právě symetrie [2][3].

Symetrie v SAT formulích lze chápat jako permutace proměnných, které zachovávají pravdivost celé formule. Jako hlavní příklad symetrií v této práci budeme uvažovat záměnu ekvivalentních objektů ve vstupní úloze. Pro lepší představu si jako jednoduchý příklad uvedeme problém holubníků. Zde je cílem určit, zda je možné rozmístit zadaný počet holubů do zadaného počtu holubníků tak, že v každém holubníku bude nejvýše jeden holub. Na jednotlivé holuby ani holubníky nemáme žádné speciální požadavky, takže například pro 5 holubníků a 1 holuba máme 5 různých řešení, protože nezáleží na tom, který holubník vybereme. Přitom z jednoho řešení na druhé se můžeme dostat permutací označení jednotlivých holubníků, protože jednotlivé holubníky jsou ekvivalentní. V tomto smyslu jsou i samotná řešení ekvivalentní a SAT řešiči by stačilo zkontrolovat jediné z nich, aby přišel na to, jestli jsou všechna pravdivá, nebo nepravdivá. Avšak právě podobné permutované varianty stejného řešení SAT řešiče často zbytečně procházejí mnohokrát během jednoho řešení, čímž se dramaticky zvětšuje prohledávací prostor. V ideálním případě SAT řešič z každé třídy ekvivalencí řešení zkontroluje vždy nejvýše jedno řešení. K tomuto stavu se chceme přidáváním klauzulí co nejvíce přiblížit a říkáme mu úplné rozbití symetrie.

Úplné rozbití všech symetrií je ale v realitě většinou příliš náročné [2]. SAT řešiče sice v takovém případě zpravidla prohledávají menší prostor, ale samotné generování všech potřebných klauzulí pro rozbíjení symetrií zabere příliš mnoho času. Musíme tedy generovat určitou podmnožinu klauzulí pro rozbíjení symetrií, která ale stále rozbije dostatečné množství symetrií. Tomuto přístupu říkáme částečné rozbití symetrie.

Jedním z nejčastěji používaných způsobů rozbíjení symetrií je **Lex-Leader** [2][4]. Zde pro každou třídu ekvivalencí řešení definujeme jedno jako kanonické řešení. Toho docílíme přidáním omezujících podmínek, které splňuje pouze kanonické řešení. Základem je nejdříve si zvolit uspořádání na proměnných. Z tohoto uspořádání odvodíme uspořádání na celých ohodnoceních stejně jako u lexikografického uspořádání. Máme tedy

$$x_1 \prec x_2 \prec \dots \prec x_n$$

a pro každé ohodnocení x a každou permutaci π přidáme omezující podmínku

$$x \leq_{\text{lex}} \pi(x).$$

Vybrané kanonické řešení je vždy lexikograficky nejmenší a tím pádem jako jediné z dané třídy ekvivalencí bude splňovat všechny přidané podmínky. Tímto zřejmě dosáhneme úplného rozbití symetrií. Použití všech permutací však pro n proměnných vede k časové složitosti alespoň $\Omega(n!)$, což je pro praktické použití nepřijatelné [2]. Z tohoto důvodu se budeme zaměřovat na částečné rozbití symetrií, kdy se omezující podmínky generují pouze pro vybranou podmnožinu permutací.

1 Teoretická část

1.1 Základní definice

Jelikož se budeme zabývat symetriemi, hlavní objekty, na které se budeme soustředit, budou samozřejmě podgrupy symetrické grupy. Prvky, které budeme permutovat, budou proměnné ze zadaných SAT formulí. Ty můžeme reprezentovat jednoduše jako čísla $\{1, \dots, n\}$, takže symetrickou grupu na těchto prvcích můžeme značit S_n .

Budeme předpokládat, že symetrie zadaného problému známe již ze zadání. Je to z toho důvodu, že budeme chtít navázat na jednu konkrétní informaci z nástrojů jako je **BreakID** nebo **Satsuma** [5][6]. Tyto nástroje dostanou na vstupu SAT formuli a snaží se v ní detekovat určité typy symetrií. Pokud symetrii detekují, tím pádem najdou určitou podgrupu $G \subseteq S_n$, rozbíjí symetrie generováním **Lex-Leader** podmínek $x \leq_{\text{lex}} \pi(x)$ pro π z množiny generátorů G , které získají při detekci symetrie [5][6]. Zmíněnou informací je, že takto získané klauzule dávají velmi dobré částečné rozbití symetrie, ale ani samotným autorům těchto nástrojů není jasné, proč tomu tak je. V této práci chceme tedy zkoumat, jestli vlastnost, že určitá množina permutací generuje celou podgrupu odpovídající symetrii daného problému, znamená, že s ní dosáhneme lepšího částečného rozbití symetrie. Pro úplnost si na jednoduchém příkladu ukážeme, že množinou generující danou grupu permutací nemusíme dosáhnout úplného rozbití symetrie, což by někoho na první pohled mohlo u generátorů napadnout.

Příklad. Použijeme příklad již zmíněný v úvodu. Uvažujme tedy problém holubníku s pěti holubníky a jedním holubem. Máme tedy celkem pět proměnných, které budeme nyní pro přehlednost značit x_1, x_2, x_3, x_4, x_5 . Hodnoty těchto proměnných mohou být pouze 1 nebo 0 a hodnota 1 znamená, že holub je v holubníku odpovídajícímu danému indexu. Z našich pěti proměnných tedy vždy musí právě jedna nabývat hodnoty 1 a právě čtyři musí nabývat hodnoty 0. Holubníky jsou si ekvivalentní a tím pádem můžeme proměnné libovolně permutovat. Pracujeme tedy s grupou S_5 (kde permutace indexů odpovídá permutaci proměnných s těmito indexy). Pro tu vezmeme množinu generátorů $\{(1,2), (1,5,4,3,2)\}$. Označme si je $\pi_1 := (1,2)$ a $\pi_2 := (1,5,4,3,2)$. Nyní dostaneme dvě podmínky:

$$\begin{aligned} x = (x_1, x_2, x_3, x_4, x_5) &\leq_{\text{lex}} (x_2, x_1, x_3, x_4, x_5) = \pi_1(x) \\ x = (x_1, x_2, x_3, x_4, x_5) &\leq_{\text{lex}} (x_2, x_3, x_4, x_5, x_1) = \pi_2(x) \end{aligned}$$

Dosazením všech možných ohodnocení dostaneme (první sloupec představuje dosazení pro π_1 a druhý sloupec pro π_2):

$$\begin{array}{ll} (1,0,0,0,0) \leq_{\text{lex}} (0,1,0,0,0) & (1,0,0,0,0) \leq_{\text{lex}} (0,0,0,0,1) \\ (0,1,0,0,0) \leq_{\text{lex}} (1,0,0,0,0) & (0,1,0,0,0) \leq_{\text{lex}} (1,0,0,0,0) \\ (0,0,1,0,0) \leq_{\text{lex}} (0,0,1,0,0) & (0,0,1,0,0) \leq_{\text{lex}} (0,1,0,0,0) \\ (0,0,0,1,0) \leq_{\text{lex}} (0,0,0,1,0) & (0,0,0,1,0) \leq_{\text{lex}} (0,0,1,0,0) \\ (0,0,0,0,1) \leq_{\text{lex}} (0,0,0,0,1) & (0,0,0,0,1) \leq_{\text{lex}} (0,0,0,1,0) \end{array}$$

Pro nás bude vždy platit, že $0 \leq_{\text{lex}} 1$ a proměnné dříve v pořadí mají větší váhu než proměnné dále. Kanonickým řešením pro nás tedy zde je $(0,0,0,0,1)$. Pokud se nyní podíváme na nerovnosti výše, vidíme, že v obou sloupcích je neplatný první řádek. To znamená, že jsme přidáním podmínek z permutací π_1, π_2 odstranili jedno ekvivalentní řešení z pěti, a to $(1,0,0,0,0)$ (k tomu by stačilo, aby byl neplatný řádek jen v jednom ze sloupců). Pro úplné rozbití symetrie ale potřebujeme odstranit čtyři z pěti těchto ohodnocení.

Analogickým postupem můžeme ověřit, že ale například pro množinu generátorů $\{(1,2), (1,2,3,4,5)\}$ úplné rozbití symetrie dostaneme a zbyde nám jako jediné možné řešení $(0,0,0,0,1)$.

V této práci se zaměříme především na problémy, jejichž řešení lze reprezentovat binární maticí [7]. Veškeré postupy je možné přímočaře zobecnit i do vícerozměrných problémů, jen se zpravidla dramaticky zvedne počet proměnných, symetrií a celkově náročnost řešení problému.

Definice 1. *Nechť A je $m \times n$ matice. Pak $\text{vec}(A)$ bude značit vektor délky $m \cdot n$, který získáme konkatenací všech řádků matice za sebe.*

Definice 2. *Nechť A, B jsou $m \times n$ binární matice a nechť $a := \text{vec}(A)$, $b := \text{vec}(B)$. Pak budeme uvažovat následující uspořádání $\leq$ definované jako:*

$$A \leq B \text{ právě tehdy, když } a \leq_{\text{lex}} b$$

Spíše než abychom ale pracovali s již známými maticemi řešení, budeme chtít lexikograficky menší řešení hledat. Místo binárních matic budou tedy matice obsahovat proměnné a pro každé ohodnocení těchto proměnných dostaneme konkrétní binární matici. Uspořádání $\leq$ z předchozí definice nám poté bude dávat podmínky, které musí ohodnocení proměnných splňovat.

Příklad. Vraťme se opět k problému holubníku. Nyní uvažujme, že máme 3 holuby a 3 holubníky. Vytvoříme si matici H , ve které budou řádky odpovídat jednotlivým holubům a sloupce holubníkům. Prvky $h_{i,j}$ matice nám pak říkají, jestli i -tý holub je v j -tém holubníku. Základní podmínky, které musíme zakódovat do SAT klauzulí, jsou, že každý holub musí být alespoň v jednom holubníku a v každém holubníku je nejvýše jeden holub. Toto kódování si popíšeme později. Nyní je podstatné, že máme následující matici:

$$H = \begin{pmatrix} x_1 & x_2 & x_3 \\ x_4 & x_5 & x_6 \\ x_7 & x_8 & x_9 \end{pmatrix}$$

H musí z výše uvedených podmínek mít v každém řádku alespoň jednu 1 a v každém sloupci nejvýše jednu 1. Již víme, že holubníky můžeme libovolně permutovat. Stejně tak zřejmě můžeme permutovat i holuby. V tomto případě tedy pracujeme s grupou $S_3 \times S_3 \subseteq S_9$. Vezmeme-li například pro řádky (holuby) permutaci $\pi_1 = (1,2,3)$ a pro sloupce (holubníky) permutaci $\pi_2 = (2,1,3)$, dostaneme pro $\pi := (\pi_1, \pi_2)$

$$\pi(H) = \begin{pmatrix} x_8 & x_9 & x_7 \\ x_2 & x_3 & x_1 \\ x_5 & x_6 & x_4 \end{pmatrix}$$

a přidáme podmínku $H \leq \pi(H)$. Což znamená

$$\begin{aligned} \text{vec}(H) &\leq_{\text{lex}} \text{vec}(\pi(H)) \text{ ,neboli} \\ (x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9) &\leq_{\text{lex}} (x_8, x_9, x_7, x_2, x_3, x_1, x_5, x_6, x_4). \end{aligned}$$

Dále už bychom mohli postupovat analogicky prvnímu příkladu s jedním holubem a pěti holubníky.

V závěru této sekce si pro úplnost ještě uvedeme a dokážeme lemma o zachování splnitelnosti při přidání **Lex-Leader** podmínek. Pro přehlednější zápis zavede pojem *permutace, která odpovídá symetrii formule*. Tímto pojmem budeme rozumět takovou permutaci proměnných, která zachovává množinu řešení formule. Aplikace této permutace na libovolné splňující ohodnocení tedy opět vede na splňující ohodnocení.

Lemma 1 (Zachování splnitelnosti při přidání **Lex-Leader** podmínek). *Nechť F je booleovská formule a necht P je množina permutací proměnných, které odpovídají symetriím formule F . Pak je formule F splnitelná právě tehdy, když je splnitelná i formule*

$$F \wedge \bigwedge_{\pi \in P} (x \leq_{\text{lex}} \pi(x)),$$

kde x je vektor proměnných.

Důkaz. Necht α je libovolné splňující ohodnocení formule F . Protože prvky množiny P jsou permutacemi odpovídajícími symetriím formule F , jsou takovými permutacemi také všechny prvky grupy $H = \langle P \rangle$. Každé ohodnocení z

$$H(\alpha) = \{\sigma(\alpha) \mid \sigma \in H\}$$

tedy rovněž splňuje formuli F . Jelikož je množina $H(\alpha)$ konečná, existuje v ní lexikograficky nejmenší prvek (kanonické řešení). Označme jej α_{kan} . Pro každou permutaci $\pi \in P$ platí

$$\pi(\alpha_{kan}) \in H(\alpha),$$

a z minimality prvku α_{kan} vzhledem k lexikografickému uspořádání proto dostáváme

$$\alpha_{kan} \leq_{\text{lex}} \pi(\alpha_{kan}).$$

Ohodnocení α_{kan} tedy splňuje všechny přidané **Lex-Leader** podmínky, a zároveň splňuje i původní formuli F . Formule

$$F \wedge \bigwedge_{\pi \in P} (\alpha \leq_{\text{lex}} \pi(\alpha))$$

je proto splnitelná.

Opačná implikace platí triviálně, protože libovolné ohodnocení splňující formuli $F \wedge \bigwedge_{\pi \in P} (\alpha \leq_{\text{lex}} \pi(\alpha))$ splňuje i formuli F . Navíc alespoň jedno takové ohodnocení musí existovat z předpokladu, že je formule splnitelná. $\square$

1.2 Direktní součin grup

V této sekci uvedeme jednoduchý výsledek z teorie grup, který v experimentální části využíváme. Již v příkladě na konci předchozí sekce jsme pracovali s grupou $S_3 \times S_3 \subseteq S_9$. $S_3 \times S_3$ zde značí direktní součin grup, který je většinou čtenářů pravděpodobně známý. Přesto si pro úplnost direktní součin v této sekci zadefinujeme a dokážeme si, že pokud známe množiny generující jednotlivé grupy, můžeme jednoduše získat i množinu generující direktní součin těchto grup.

V implementaci našeho nástroje, kterou popíšeme v pozdější části práce, často nepracujeme přímo s permutacemi všech proměnných, ale postupujeme následovně. Pokud například matice proměnných pro uvažovaný problém obsahuje řádkové a sloupcové symetrie (jako právě matice pro problém holubníku), máme k dispozici množinu generátorů grupy řádkových symetrií a množinu generátorů grupy sloupcových symetrií. Každou z těchto množin rozšíříme o neutrální prvek příslušné grupy. Poté pracujeme s kartézským součinem těchto množin. Jelikož chceme mimo jiné zkoumat, jestli vlastnost, že množina generuje danou grupu, má vliv na účinnost rozbíjení symetrií, je třeba ukázat, že kartézský součin popsanych množin generuje direktní součin daných grup.

Definice 3. *Direktním součinem grup G_i pro $i = 1, \dots, n$ rozumíme grupu $G = G_1 \times \dots \times G_n$, jejíž operace jsou definovány po složkách.*

Že výsledná grupa G je opravdu grupou je možné ověřit přímým ověřením všech axiomů grup.

V následujícím tvrzení pro odlišení zavedeme značení $\oplus_G$ pro grupovou operaci grupy G a e_G pro neutrální prvek grupy G .

Tvrzení 2 (Množina generující direktní součin grup). *Nechť G, H jsou grupy a mějme množiny $A \subseteq G, B \subseteq H$ takové, že $G = \langle A \rangle$ a $H = \langle B \rangle$. Pak*

$$G \times H = \langle ((A \cup \{e_G\}) \times (B \cup \{e_H\})) \rangle.$$

Důkaz. Mějme prvek $(a, b) \in G \times H$. Pak $(a, b) = (a, e_H) \oplus_{G \times H} (e_G, b)$. Z $G = \langle A \rangle$ a $H = \langle B \rangle$ víme, že existují $a_1, \dots, a_n \in A$ a $b_1, \dots, b_m \in B$ taková, že $a = a_1 \oplus_G \dots \oplus_G a_n$ a také $b = b_1 \oplus_H \dots \oplus_H b_m$. Z toho plyne, že

$$(a, e_H) = (a_1 \oplus_G \dots \oplus_G a_n, e_H) = (a_1, e_H) \oplus_{G \times H} \dots \oplus_{G \times H} (a_n, e_H)$$

a podobně

$$(e_G, b) = (e_G, b_1 \oplus_H \dots \oplus_H b_m) = (e_G, b_1) \oplus_{G \times H} \dots \oplus_{G \times H} (e_G, b_m).$$

Celkem tedy

$$(a, b) = (a_1, e_H) \oplus_{G \times H} \dots \oplus_{G \times H} (a_n, e_H) \oplus_{G \times H} (e_G, b_1) \oplus_{G \times H} \dots \oplus_{G \times H} (e_G, b_m).$$

V každé z dvojic na pravé straně rovnice je na první pozici prvek množiny $(A \cup \{e_G\})$ a na druhé pozici prvek množiny $(B \cup \{e_H\})$. Jelikož lze takto vyjádřit libovolný prvek $G \times H$, tak $((A \cup \{e_G\}) \times (B \cup \{e_H\}))$ generuje $G \times H$. $\square$

1.3 Kódování podmínek

V této části se podíváme blíže na to, jak budeme kódovat podmínky $\leq_{\text{lex}}$ jako SAT klauzule. Využijeme k tomu takzvané *vzory* (anglicky *patterns*) ze [8], kde byly využity výhradně pro grafové problémy. Idea je taková, že pro podmínku $x \leq_{\text{lex}} \pi(x)$ budeme naopak postupně sestavovat podmínku $\pi(x) <_{\text{lex}} x$, kterou poté znegujeme. Postupným sestavováním se myslí, že si v každém kroku zafixujeme pozici ve vektorech, na které se mají poprvé lišit (respektive první pozice, na které je proměnná v $\pi(x)$ ostře menší než proměnná v x). To nám zároveň říká, že na všech předchozích pozicích se vektory musí rovnat. Z těchto vlastností pak odvodíme hledaný vzor.

Definice 4. *Nechť $a = (a_1, \dots, a_n)$ a $b = (b_1, \dots, b_n)$. Pak řekneme, že a je menší než b na i -té pozici, pokud*

$$(a_1 = b_1) \wedge \dots \wedge (a_{i-1} = b_{i-1}) \wedge (a_i < b_i).$$

Tuto vlastnost budeme značit $a <_{\text{lex}}^i b$.

Poznámka. Jelikož pracujeme s binárními proměnnými, $(a_i < b_i)$ znamená jednoduše $(a_i = 0) \wedge (b_i = 1)$.

Lemma 3.

$$a <_{\text{lex}} b \Leftrightarrow \bigvee_{i=1}^n a <_{\text{lex}}^i b$$

Důkaz. Pokud $a <_{\text{lex}} b$, tak musí existovat alespoň jedna pozice i taková, že $a_i < b_i$. Vezměme si za i nejmenší takovou pozici. Pak pro všechna $j < i$ musí platit $a_j = b_j$, protože pokud by $a_j > b_j$ pro nějaké $j < i$, platilo by, že $a >_{\text{lex}} b$. To plyne z toho, že v námi používaném lexikografickém uspořádání mají dřívější pozice vždy větší váhu. Našli jsme tedy i takové, že $a_i < b_i$ a zároveň $a_j = b_j$ pro všechna $j < i$. To je přesně definice $a <_{\text{lex}}^i b$.

Naopak pokud $\bigvee_{i=1}^n a <_{\text{lex}}^i b$, tak máme alespoň jedno i takové, že $a_i < b_i$ a $a_j = b_j$ pro všechna $j < i$. To ale okamžitě znamená, že $a <_{\text{lex}} b$. $\square$

Definice 5. *Nechť π je permutace a $a = (a_1, \dots, a_n)$. Pak vzorem $\text{pat}_i(\pi)$ rozumíme aplikování rovností a nerovností z $\pi(a) <_{\text{lex}}^i a$ na vektor a .*

Příklad. Uvažujme vektory z předchozího příkladu a mějme $\text{vec}(\pi(H)) <_{\text{lex}}^i \text{vec}(H)$ pro $i = 4$, neboli

$$(x_8, x_9, x_7, x_2, x_3, x_1, x_5, x_6, x_4) <_{\text{lex}}^4 (x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8, x_9).$$

Tím tedy dostáváme podmínky $(x_8 = x_1), (x_9 = x_2), (x_7 = x_3)$ a $(x_2 < x_4)$. To znamená, že $x_2 = 0$, $x_4 = 1$ a zároveň také z druhé rovnosti $x_9 = 0$. Pro odlišení dále označíme $A := x_1 (= x_8)$ a $B := x_3 (= x_7)$. Dosazením (aplikováním) těchto hodnot do $\text{vec}(H)$ dostaneme

$$\text{pat}_4(\pi) = (A, 0, B, 1, x_5, x_6, B, A, 0).$$

Nyní pokud libovolný vektor v odpovídá tomuto vzoru, tak o něm automaticky víme, že $v <_{\text{lex}} \text{vec}(H)$. Jelikož A, B, x_5, x_6 mohou být libovolná, pokryli jsme tímto vzorem celkem $2^4 = 16$ různých vektorů.

1.4 Popis problémů a jejich kódování

1.4.1 Problém holubníku (Pigeonhole principle)

Jako první se podíváme na již několikrát zmiňovaný problém holubníku. Princip holubníku nám říká, že pokud máme m předmětů (holubů), chceme je rozmístit do n kontejnerů (holubníků) a navíc $m > n$, pak musí alespoň jeden kontejner obsahovat více než jeden předmět. Přestože se toto tvrzení zdá být zřejmé, pomáhá vysvětlit spoustu zdaleka ne tak zřejmých tvrzení. Tento princip přeformulujeme do tvaru problému následovně. Pro zadaný počet holubů m a počet holubníků n musíme holuby rozmístit do holubníků tak, aby každý holub byl alespoň v jednom holubníku a aby v každém holubníku byl nejvýše jeden holub. Většinou se uvažuje $m = n + 1$, takže je problém nesplnitelný. Přestože se výsledek opět zdá být zřejmý, pro SAT řešiče je řešení tohoto problému bez rozbíjení symetrií velmi náročné [9]. Proto se tento problém velmi často využívá jako základní benchmark právě pro testování rozbíjení symetrií.

Kódování tohoto problému do SAT formule je přímočaré. Stejně jako v jednom z předchozích příkladů budeme mít matici binárních proměnných, kde řádky budou odpovídat holubům a sloupce holubníkům. Máme tedy $m \times n$ matici. Proměnné jsou x_{in+j+1} , kde i je řádek (holub) a j je sloupec (holubník) a $x_{in+j+1} = 1$, pokud i -tý holub je umístěn do j -tého holubníku, a $x_{in+j+1} = 0$ jinak. Řádky a sloupce budeme v této sekci vždy indexovat od 0. Kódování podmínek pak vypadá následovně:

Poznámka. (Problém holubníku jako SAT formule) Každý holub v alespoň jednom holubníku:

$$\bigwedge_{i=0}^{m-1} (x_{in+1} \vee x_{in+2} \vee \cdots \vee x_{in+n})$$

V každém holubníku nejvýše jeden holub:

$$\bigwedge_{j=0}^{n-1} \bigwedge_{0 \leq i < k \leq m-1} (\neg x_{in+j+1} \vee \neg x_{kn+j+1})$$

Jak již bylo také zmíněno, symetrie zde leží v tom, že můžeme mezi sebou permutovat zvlášť holuby a zvlášť holubníky. Můžeme tedy permutovat buď řádky, sloupce, nebo obojí nezávisle na sobě.

1.4.2 Problém N královen (N-queens problem)

Dalším z vybraných problémů je známý problém N královen. Zde dostaneme na vstupu jediný parametr n a za úkol máme rozmístit n šachových figurek královen na $n \times n$ šachovnici tak, aby se žádné dvě z nich vzájemně neohrožovaly. Řešení tohoto problému pro SAT řešiče není až tak náročné i bez rozbíjení symetrií a jedno řešení zvládají rychle najít i pro relativně velká n . Avšak celkový počet řešení se zvyšujícím se n stoupá dramaticky a my tím pádem můžeme dobře sledovat, o kolik se celkový počet řešení po rozbíjení symetrií sníží.

Kódování problému N královen do SAT formule také není složité. Vytvoříme $n \times n$ matici binárních proměnných, kde jednotlivé pozice odpovídají pozicím na šachovnici. Pro i -tý řádek a j -tý sloupec pak $x_{in+j} = 1$, pokud jsme umístili

královnu na pozici (i, j) , a $x_{in+j} = 0$ jinak. Nejjednodušší kódování pak dostaneme pro podmínky právě jedné královny v každém řádku, sloupci a na každé diagonále následovně:

Poznámka. (Problém N královen jako SAT formule) V každém řádku alespoň jedna královna:

$$\bigwedge_{r=0}^{n-1} \left(\bigvee_{c=0}^{n-1} x_{rn+c+1} \right)$$

V každém řádku nejvýše jedna královna:

$$\bigwedge_{r=0}^{n-1} \bigwedge_{0 \leq c_1 < c_2 \leq n-1} (\neg x_{rn+c_1+1} \vee \neg x_{rn+c_2+1})$$

V každém sloupci alespoň jedna královna:

$$\bigwedge_{c=0}^{n-1} \left(\bigvee_{r=0}^{n-1} x_{rn+c+1} \right)$$

V každém sloupci nejvýše jedna královna:

$$\bigwedge_{c=0}^{n-1} \bigwedge_{0 \leq r_1 < r_2 \leq n-1} (\neg x_{r_1n+c+1} \vee \neg x_{r_2n+c+1})$$

Nejvýše jedna královna diagonálně směrem doprava nahoru:

$$\bigwedge_{d=-(n-1)}^{n-1} \bigwedge_{\substack{0 \leq r_1 < r_2 \leq n-1 \\ 0 \leq r_1-d < n, 0 \leq r_2-d < n}} (\neg x_{r_1n+(r_1-d)+1} \vee \neg x_{r_2n+(r_2-d)+1})$$

Nejvýše jedna královna diagonálně směrem doprava dolů:

$$\bigwedge_{s=0}^{2n-2} \bigwedge_{\substack{0 \leq r_1 < r_2 \leq n-1 \\ 0 \leq s-r_1 < n, 0 \leq s-r_2 < n}} (\neg x_{r_1n+(s-r_1)+1} \vee \neg x_{r_2n+(s-r_2)+1})$$

Kvůli podmínkám na diagonály u tohoto problému zřejmě nemůžeme libovolně permutovat řádky a sloupce. Avšak problém symetrie samozřejmě obsahuje. Z jednoho řešení můžeme získat několik dalších ekvivalentních řešení pomocí rotací a reflexí celé šachovnice. Zastoupení dalšího typu symetrií je dalším z důvodů zařazení problému N královen.

1.4.3 Problém nalezení matice s řádky a sloupci se zadanými součty

Pro následující problém může už být zadání zajímavější. Na vstupu dostaneme množinu čísel K a množinu čísel L , kde $m := |K|$, $n := |L|$, prvky K budeme značit k_r pro $r \in \{0, \dots, m-1\}$ a prvky L budeme značit l_c pro $c \in \{0, \dots, n-1\}$. Každý prvek K odpovídá zadání pro jeden řádek a každý prvek L odpovídá zadání pro jeden sloupec. Máme tedy $m \times n$ matici binárních proměnných. Naším úkolem pak je přiřadit proměnným hodnoty 0 a 1 tak, aby součet hodnot v r -tém řádku byl právě k_r , a aby součet hodnot v c -tém sloupci byl právě l_c .

Kódování je v tomto případě už také složitější. Podmínky na právě k_r jedniček v řádku a právě l_c jedniček ve sloupci se sice ve své podstatě podobají podmínkám na právě jednu jedničku v řádku/sloupci z předchozích problémů, ale počet klauzulí je zpravidla výrazně vyšší, protože přidáváme klauzuli pro každou kombinaci určitého počtu proměnných. Existují efektivnější kódování využívající pomocné proměnné (například [10]), ale těmi se v této práci nebudeme zabývat, jelikož problémy zde slouží především k testování. Hlavní ideou práce je, že na vstupu dostaneme SAT formuli a typ symetrie, se kterými pracujeme dále. Tím pádem nemáme zaručeno, že kódování je jakkoliv optimalizované, a zároveň ho sami optimalizovat nemůžeme, jelikož nevíme, co za problém SAT formule představuje. Námi použité kódování pak bude vypadat následovně:

Poznámka. (Problém nalezení matice s řádky a sloupce se zadanými součty jako SAT formule) Alespoň k_r jedniček v řádku r :

$$\bigwedge_{r=0}^{m-1} \bigwedge_{\substack{S \subseteq \{0, \dots, n-1\} \\ |S|=n-k_r+1}} \left(\bigvee_{c \in S} x_{rn+c+1} \right)$$

Nejvýše k_r jedniček v řádku r :

$$\bigwedge_{r=0}^{m-1} \bigwedge_{\substack{S \subseteq \{0, \dots, n-1\} \\ |S|=k_r+1}} \left(\bigvee_{c \in S} \neg x_{rn+c+1} \right)$$

Alespoň l_c jedniček ve sloupci c :

$$\bigwedge_{c=0}^{n-1} \bigwedge_{\substack{T \subseteq \{0, \dots, m-1\} \\ |T|=m-l_c+1}} \left(\bigvee_{r \in T} x_{rn+c+1} \right)$$

Nejvýše l_c jedniček ve sloupci c :

$$\bigwedge_{c=0}^{n-1} \bigwedge_{\substack{T \subseteq \{0, \dots, m-1\} \\ |T|=l_c+1}} \left(\bigvee_{r \in T} \neg x_{rn+c+1} \right)$$

Symetrie pro tento problém jsou také řádkové a sloupkové jako u problému holubníku, avšak permutovat mezi sebou můžeme jen řádky se stejnými požadovanými součty a stejně tak jen sloupce se stejnými požadovanými součty.

1.4.4 Problém latinských čtverců

Další z velmi známých problémů je problém latinských čtverců. Cílem je pro vstupní parametr n rozmístit do $n \times n$ matice hodnoty $1, \dots, n$ (respektive v našem případě hodnoty $0, \dots, n-1$, abychom indexovali od 0 stejně jako řádky a sloupce) tak, aby se v každém řádku a v každém sloupci vyskytovala každá hodnota právě jednou. Jelikož proměnné mohou nabývat pouze 0, nebo 1, je tento problém na první pohled trojrozměrný. Nejjednodušší reprezentací se nabízí použití proměnných $x_{r,c,v}$, kde každá tato proměnná říká, jestli na pozici v r -tém řádku a v c -tém sloupci se vyskytuje hodnota v . My bychom ale chtěli opět pracovat

s dvourozměrnou maticí. Proto si pro každou hodnotu v vrstvy trojrozměrné matice rozdělíme na n binárních matic $n \times n$ a ty poskládáme pod sebe do jedné dvourozměrné matice. Výsledkem tedy bude $n^2 \times n$ binární matice, kde proměnná $x_{vn^2+rn+c+1}$ odpovídá proměnné $x_{r,c,v}$ výše. Pro přehlednost však v rámci této sekce zůstaneme u zápisu pomocí proměnných ve tvaru $x_{r,c,v}$.

Kódování pak dostaneme již dobře známým postupem, kdy si podmínky pro právě jednu hodnotu na každé pozici, každou hodnotu právě jednou v každém řádku a každou hodnotu právě jednou v každém sloupci rozdělíme na část „alespoň jednou” a na část „nejvýše jednou”. Ty zapíšeme analogicky zápisu z předchozích problémů. Kódování pak vypadá následovně:

Poznámka. (Problém latinských čtverců jako SAT formule) Alespoň jedna hodnota na každé pozici:

$$\bigwedge_{r=0}^{n-1} \bigwedge_{c=0}^{n-1} \left(\bigvee_{v=0}^{n-1} x_{r,c,v} \right)$$

Nejvýše jedna hodnota na každé pozici:

$$\bigwedge_{r=0}^{n-1} \bigwedge_{c=0}^{n-1} \bigwedge_{0 \leq v_1 < v_2 < n} (\neg x_{r,c,v_1} \vee \neg x_{r,c,v_2})$$

Každá hodnota alespoň jednou v každém řádku:

$$\bigwedge_{r=0}^{n-1} \bigwedge_{v=0}^{n-1} \left(\bigvee_{c=0}^{n-1} x_{r,c,v} \right)$$

Každá hodnota nejvýše jednou v každém řádku:

$$\bigwedge_{r=0}^{n-1} \bigwedge_{v=0}^{n-1} \bigwedge_{0 \leq c_1 < c_2 < n} (\neg x_{r,c_1,v} \vee \neg x_{r,c_2,v})$$

Každá hodnota alespoň jednou v každém sloupci:

$$\bigwedge_{c=0}^{n-1} \bigwedge_{v=0}^{n-1} \left(\bigvee_{r=0}^{n-1} x_{r,c,v} \right)$$

Každá hodnota nejvýše jednou v každém sloupci:

$$\bigwedge_{c=0}^{n-1} \bigwedge_{v=0}^{n-1} \bigwedge_{0 \leq r_1 < r_2 < n} (\neg x_{r_1,c,v} \vee \neg x_{r_2,c,v})$$

U latinských čtverců můžeme zřejmě nezávisle na sobě permutovat řádky, sloupce a i hodnoty, které na pozice zapisujeme. U tohoto problému je dobře známé, že i pro malá n je celkový počet řešení obrovský. Například pro $n = 5$ existuje 161280 řešení, ale pro $n = 6$ jich existuje již 812851200 (počty řešení lze ověřit v posloupnosti A002860 z [11]). Tyto počty jsou však počítány bez jakéhokoliv rozbíjení symetrií a díky symetriím v řádcích, sloupcích i v hodnotách můžeme očekávat, že jich je tím pádem velmi velké množství ekvivalentních. V naší reprezentaci $n^2 \times n$ binárních matic musíme řádky permutovat vždy zvlášť v každé z n disjunktích $n \times n$ binárních podmatic (celá podmatice musí vždy odpovídat pouze jedné z hodnot a pokud permutujeme řádky u jedné z podmatic, musíme je stejně permutovat i u všech podmatic odpovídajícím ostatním hodnotám) a symetrie na hodnotách rozbíjíme permutacemi celých $n \times n$ binárních podmatic představujících jednotlivé hodnoty. Sloupce můžeme permutovat běžně bez žádných speciálních omezení.

1.4.5 Problém společenských golfistů

Posledním problémem, kterým se budeme zabývat je problém společenských golfistů. Zde dostaneme na vstupu počet týdnů W , počet skupin G a počet hráčů v každé skupině p' . Celkový počet hráčů si označíme P a musí tedy platit $P = p' * G$. Za úkol pak máme rozvrhnout hry golfu na W týdnů tak, aby každý týden hrál každý hráč v některé ze skupin, a zároveň aby žádná dvojice hráčů nebyla společně ve stejné skupině za všechny týdny více než jednou. Tento problém je opět trojrozměrný. My však opět použijeme zápis dvourozměrnou binární maticí z problému latinských čtverců. Uvažujeme $P \times G$ binární podmatice, kterých pod sebe poskládáme W . Výslednou maticí je $(P \cdot W) \times G$ binární matice, ve které proměnné $x_{wPG+pG+g+1}$ říkají, jestli hráč p hraje ve skupině g ve w -tý týden. Pro přehlednost v této sekci zapisujeme tyto proměnné podobně jako v předchozím problému ve tvaru $x_{w,p,g}$.

Kódování se převážně opět podobá kódováním předchozích problémů. Zásadně se odlišuje pouze podmínkou na to, že se žádný pár hráčů nemůže potkat dvakrát. Ta je zde podobně jako u problému nalezení matice s řádky a sloupci se zadanými součty zakódována nejjednodušším způsobem, přestože existují efektivnější kódování. Výsledné kódování je pak následující:

Poznámka. (Problém společenských golfistů jako SAT formule) Každý hráč v alespoň jedné skupině každý týden:

$$\bigwedge_{w=0}^{W-1} \bigwedge_{p=0}^{P-1} \left(\bigvee_{g=0}^{G-1} x_{w,p,g} \right)$$

Každý hráč v nejvýše jedné skupině každý týden:

$$\bigwedge_{w=0}^{W-1} \bigwedge_{p=0}^{P-1} \bigwedge_{0 \leq g_1 < g_2 < G} (\neg x_{w,p,g_1} \vee \neg x_{w,p,g_2})$$

Každá skupina má alespoň p' hráčů každý týden:

$$\bigwedge_{w=0}^{W-1} \bigwedge_{g=0}^{G-1} \bigwedge_{\substack{S \subseteq P \\ |S|=p'+1}} \left(\bigvee_{p \in S} x_{w,p,g} \right)$$

Každá skupina má nejvýše p' hráčů každý týden:

$$\bigwedge_{w=0}^{W-1} \bigwedge_{g=0}^{G-1} \bigwedge_{\substack{S \subseteq P \\ |S|=p'+1}} \left(\bigvee_{p \in S} \neg x_{w,p,g} \right)$$

Žádný pár hráčů se nepotká dvakrát:

$$\bigwedge_{0 \leq i < j < P} \bigwedge_{0 \leq w_1 < w_2 < W} \bigwedge_{g_1=0}^{G-1} \bigwedge_{g_2=0}^{G-1} (\neg x_{w_1,i,g_1} \vee \neg x_{w_1,j,g_1} \vee \neg x_{w_2,i,g_2} \vee \neg x_{w_2,j,g_2})$$

Ekvivalentní jsou si zde vzájemně hráči, skupiny i týdny. Všechny parametry tedy opět můžeme nezávisle na sobě libovolně permutovat a získáme ekvivalentní řešení. Permutace v naší dvourozměrné reprezentaci opět musíme provádět analogicky případu u problému latinských čtverců.

2 Technická dokumentace

2.1 Generování vstupních instancí

Přestože hlavním cílem této práce není generování SAT kódování jednotlivých problémů, ale generování klauzulí pro rozbíjení symetrií nad již existujícími SAT formulemi, bylo pro experimentální část nutné vytvořit také nástroj pro generování testovacích instancí. Ten slouží především k automatizovanému vytváření vstupních formulí pro vybrané problémy popsané v předchozí kapitole.

Program je implementován jako samostatný skript s názvem `geninput.py` spouštěný z příkazové řádky. Při spuštění musíme zadat parametry pro všechny použité problémy. Samotné generování SAT formulí přímo odpovídá kódováním uvedeným v sekci 1.3. Pro každý z problémů je implementována samostatná funkce vytvářející množinu klauzulí podle příslušných podmínek. Jelikož se jedná pouze o technickou realizaci již popsaných formulací, nebudeme zde jednotlivé algoritmy detailně rozebírat.

Určitou výjimku představuje problém nalezení matice s řádky a sloupci se zadanými součty. Obecně lze zadat libovolné posloupnosti požadovaných řádkových a sloupcových součtů, jejichž celkové součty jsou shodné. Pro účely experimentů však byly tyto hodnoty generovány náhodně. Nejprve jsou nezávisle vygenerovány náhodné součty pro zadaný počet řádků a sloupců. Následně je podle potřeby přidán jeden dodatečný řádek nebo sloupec tak, aby byl celkový součet řádkových a sloupcových požadavků stejný. Tím je zajištěno, že vzniklá instance splňuje základní nutnou podmínku existence řešení.

Vygenerované SAT formule jsou ukládány ve formátu DIMACS CNF [12], který představuje standard pro výměnu SAT instancí mezi jednotlivými nástroji. Soubor obsahuje hlavičku určující počet proměnných a klauzulí a dále seznam jednotlivých klauzulí. Každá klauzule je zapsána jako posloupnost celých čísel reprezentujících literály a je ukončena hodnotou 0. Literály jsou celými čísly reprezentovány v celé implementaci nástroje. Kladná hodnota označuje proměnnou, záporná hodnota její negaci. Pro každý problém je vygenerován samostatný soubor tvaru `input{problém}.cnf`.

Formát DIMACS zároveň umožňuje používat komentáře zapsané v souboru řádkem začínajícím znakem „c“. Této možnosti je v implementaci využito k ukládání doplňujících informací, které nejsou součástí samotné SAT formule, ale jsou potřebné při následném generování klauzulí pro rozbíjení symetrií. Do komentářů jsou ukládány zejména údaje o rozměrech binární matice reprezentující řešení problému a o typu uvažované symetrie. Dále jsou v komentářích ukládány parametry zadaného problému, které se využívají pro pojmenování výstupních souborů po generování klauzulí pro rozbíjení symetrií.

Informace předávané v komentářích jsou pro nás klíčové, jelikož předpokládáme, že známe symetrie zadaného problému, ale nástroj pro generování klauzulí pro rozbíjení symetrií ze samotné zadané formule tyto informace není schopen získat.

Při implementaci byla využita třída `CNF` z knihovny `pysat` [13].

2.2 Generování klauzulí

Hlavní část implementace je soustředěna v souboru `sym.py`, který představuje vlastní nástroj pro generování klauzulí pro rozbíjení symetrií. Na rozdíl od programu popsaného v předchozí sekci již tento nástroj nepracuje s konkrétním problémem, ale pouze s jeho SAT formulací a informacemi o příslušných symetriích. Díky tomu jej lze použít pro libovolný problém, pro který známe odpovídající množinu symetrií a vhodnou reprezentaci proměnných binární maticí.

Stejně jako program pro generování vstupních instancí je i tento nástroj spouštěn z příkazové řádky. Při spuštění je potřeba zadat vstupní soubor ve formátu DIMACS, zvolenou množinu permutací, maximální hloubku generování klauzulí k_{max} a informaci o tom, zda bude výsledná formule použita pouze pro rozhodování o splnitelnosti, nebo také pro určení celkového počtu řešení.

Po načtení vstupního souboru program získá z komentářů informace uložené při generování instance. Jedná se zejména o rozměry binární matice reprezentující řešení problému a o typ symetrie, se kterým má dále pracovat. Tyto údaje jsou následně využity při vytvoření matice proměnných a při volbě odpovídajícího způsobu generování permutací. Dále jsou načteny parametry původního problému, které se využívají jen při vytváření názvů výstupních souborů.

Samotné generování klauzulí pro rozbíjení symetrií je realizováno funkcí `SymmetryClauses`, která je uvedena v Algoritmu 4. Této funkci je předána matice proměnných vygenerována funkcí `GetMatrix` (Algoritmus 1), typ symetrie, zvolená množina permutací a parametr k_{max} . Funkce si udržuje seznam již vygenerovaných klauzulí a zároveň eviduje pomocné proměnné vytvářené během převodu vzorů do CNF.

Další postup závisí na typu zadané symetrie. Podle toho, zda pracujeme se symetriemi na řádcích, sloupcích, obou dimenzích současně nebo se speciálními typy symetrií používanými u některých problémů, je postupně procházena odpovídající množina zpermutovaných matic. Pro každou takto získanou permutaci je následně původní matice porovnána s její zpermutovanou variantou, z čehož získáváme klauzule pomocí vzorů způsobem popsaným v první kapitole.

Vlastní generování klauzulí probíhá opakovaným voláním funkce `GetClauses`, která je uvedena v Algoritmu 3. Tato funkce postupně vytváří vzory odpovídající jednotlivým možnostem, jak může být zpermutované řešení lexikograficky menší než původní řešení [8]. Z těchto vzorů jsou následně odvozeny CNF klauzule reprezentující **Lex-Leader** podmínky. Všechny takto získané klauzule jsou průběžně přidávány do výsledné formule.

Funkce `SymmetryClauses` po průchodu celým cyklem už jen vrátí množinu vygenerovaných klauzulí do funkce `main`. Zde se společně s celým obsahem vstupního souboru a komentářem obsahující dobu, kterou trvalo generování klauzulí, uloží do výstupního souboru opět ve formátu DIMACS. Název tohoto souboru je ve tvaru `output{problém}_{parametry}_{typ permutace}_{kmax}.cnf` pro odlišení jednotlivých běhů s různými parametry. Tento formát je však použit jen pro testování a verze určena pro obecný vstup vytvoří soubor s názvem vstupního souboru, před který se připojí přípona `output_`.

Popsaný postup představuje pouze obecný přehled fungování implementace. Jednotlivé části algoritmu, způsob generování permutací i samotný převod vzorů na SAT klauzule budou podrobně rozebrány v následujících sekcích.

Stejně jako při generování vstupních instancí byla i při implementaci generování klauzulí pro rozbíjení symetrií využita třída CNF z knihovny `pysat` [13].

2.3 Pomocné funkce

Než přejdeme k samotnému generování klauzulí pro rozbíjení symetrií, popíšeme nejprve několik pomocných funkcí, které jsou využívány v hlavní části implementace. Jedná se především o funkce pro generování zvolených množin permutací a dále o funkce pro vytvoření matice proměnných a aplikaci permutací na tuto matici.

První skupinu tvoří funkce generující množiny permutací, ze kterých se následně vytvářejí **Lex-Leader** podmínky. Funkce **AllPerm** generuje všechny permutace množiny $\{0, \dots, n-1\}$. Tato volba odpovídá úplnému rozbití symetrie vzhledem k danému typu symetrie, kterou jsme dostali v zadání. Pro tuto množinu permutací ale můžeme potenciálně dostat řádově až $n!$ klauzulí [9], takže je použitelná jen pro velmi malá zadání.

Funkce **Transpositions** generuje všechny transpozice, tedy všechny permutace, které prohodí právě dvě pozice a ostatní ponechají beze změny. Oproti všem permutacím jde o výrazně menší množinu, jelikož není těžké nahlédnout, že počet všech transpozic na n prvcích je $\frac{n(n-1)}{2}$.

Funkce **Neighbors** generuje pouze transpozice sousedních pozic. Pro n prvků tedy vznikne množina $\{(0,1), (1,2), \dots, (n-2, n-1)\}$, která obsahuje přesně $n-1$ prvků. Tyto sousední transpozice tvoří standardní množinu generátorů symetrické grupy (S_n). Právě proto je tato množina důležitá při porovnávání vlivu generujících množin permutací na kvalitu částečného rozbití symetrie.

Funkce **Randtrans** generuje náhodně zvolenou množinu $n-1$ transpozic. Tato množina má stejnou velikost jako množina sousedních transpozic, a proto ji lze využít jako přirozené porovnání s funkcí **Neighbors**. Rozdíl je v tom, že transpozice nejsou voleny podle sousedních prvků, ale volí se náhodně.

Funkce **Nongen** generuje množinu náhodných transpozic, která negeneruje celou symetrickou grupu (S_n). Transpozic je opět vždy celkem $n-1$, takže při porovnání s **Neighbors** a **Randtrans** lze sledovat, zda je pro kvalitu částečného rozbití důležitá vlastnost, že množina permutací generuje danou grupu. Že množina negeneruje symetrickou grupu je zajištěno jednoduše tím, že se při generování jedna z permutovaných hodnot vyřadí a nemůže se objevit v žádné z transpozic.

Funkce **Gen2** vrací dvouprvkovou množinu generátorů symetrické grupy. Ta se skládá z cyklu délky n a jedné transpozice. Tato volba je zajímavá především tím, že generuje celou symetrickou grupu pomocí velmi malého počtu permutací.

Funkce **DihedralGen** je v této práci využívána především pro problém N královen. V tomto případě nelze libovolně permutovat řádky a sloupce, protože by se obecně nezachovaly diagonální podmínky. Symetrie šachovnice jsou proto reprezentovány pomocí rotace o 90° a reflexe podle svislé osy. Tyto dvě permutace generují dihedralní grupu symetrií čtverce. Tuto množinu permutací by samozřejmě bylo možné použít i pro jiné problémy s tímto typem symetrie.

Poslední speciální funkcí je **SumsNeighbors**, která je určena pro problém nalezení matice s řádky a sloupci se zadanými součty. V tomto problému nelze libovolně permutovat všechny řádky a sloupce, ale pouze ty, které mají stejný požadovaný

součet. Funkce proto nejprve rozdělí indexy podle odpovídajících hodnot součtů a následně v každé takto vzniklé skupině generuje sousední transpozice.

Druhou skupinu pomocných funkcí tvoří funkce **GetMatrix** a **GetPerm**. Funkce **GetMatrix**, uvedená v Algoritmu 1, vytvoří $(m \times n)$ matici proměnných. Proměnné jsou do matice zapisovány postupně po řádcích, tedy stejným způsobem, jakým jsme v teoretické části definovali operaci *vec*.

Funkce **GetPerm**, uvedená v Algoritmu 2, aplikuje na zadanou matici permutaci řádků a permutaci sloupců. Pro každý prvek původní matice určí jeho novou pozici podle zadaných permutací a vytvoří tak zpermutovanou matici. Tato funkce se následně používá při vytváření dvojice matic, jejichž vektorizace se porovnávají ve funkci **GetClauses**.

Algoritmus 1 Vytvoření matice proměnných

```

1: function GETMATRIX( $m, n$ )
2:    $variables \leftarrow m \times n$  nulová matice
3:    $idx \leftarrow 1$ 
4:   for  $i \leftarrow 0$  to  $m - 1$  do
5:     for  $j \leftarrow 0$  to  $n - 1$  do
6:        $variables_{i,j} \leftarrow idx$ 
7:        $idx \leftarrow idx + 1$ 
8:     end for
9:   end for
10:  return  $variables$ 
11: end function

```

Algoritmus 2 Aplikace permutací na matici

```

1: function GETPERM( $matrix, \pi_r, \pi_c$ )
2:    $m \leftarrow \#$ řádků  $matrix$ 
3:    $n \leftarrow \#$ sloupců  $matrix$ 
4:    $varperm \leftarrow m \times n$  nulová matice
5:   for  $i \leftarrow 0$  to  $m - 1$  do
6:     for  $j \leftarrow 0$  to  $n - 1$  do
7:        $varperm_{\pi_r(i), \pi_c(j)} \leftarrow matrix_{i,j}$ 
8:     end for
9:   end for
10:  return  $varperm$ 
11: end function

```

2.4 Generování klauzulí z porovnání dvou vektorů proměnných

Nejdůležitější částí implementace je funkce **GetClauses**, uvedená v Algoritmu 3. Tato funkce dostane na vstupu dva vektory proměnných a a b , kde a odpovídá vektorizaci původní matice a b vektorizaci její zpermutované varianty. Cílem je vygenerovat klauzule odpovídající podmínce $a \leq_{\text{lex}} b$.

Jak již bylo popsáno v teoretické části, místo přímého kódování této podmínky postupně konstruuujeme opačnou situaci, tedy možnost, že b je lexikograficky menší než a . Jednotlivé kroky odpovídají tomu, že b je menší než a na i -té pozici pro krok i , tedy $b <_{\text{lex}}^i a$. Pro klauzule pak použijeme negace těchto podmínek.

Funkce postupně prochází pozice $i = 1, \dots, \min(k_{\max}, n)$. Parametr $k_{\max}$ zde určuje maximální hloubku, do které se vzory generují. Pokud je tedy $k_{\max}$ menší než délka vektoru, neuvažují se všechny možné pozice, ale pouze $b <_{\text{lex}}^i a$ pro všechna $i \leq k_{\max}$. Tím lze regulovat počet vygenerovaných klauzulí.

Příklad. Uvedeme si jednoduchý příklad pro vektory proměnných

$$a = (x_1, x_2, x_3, x_4, x_5), b = (x_2, x_3, x_1, x_4, x_5).$$

Pro $k_{\max} = 1$ se použije pouze $b <_{\text{lex}}^1 a$, což z definice dává jedinou podmínku $x_2 < x_1$. Pro $k_{\max} = 3$ už používáme nerovnosti $b <_{\text{lex}}^1 a$, $b <_{\text{lex}}^2 a$, $b <_{\text{lex}}^3 a$. Ty nám z definice dávají podmínky $x_2 < x_1$ pro první pozici, $x_2 = x_1, x_3 < x_2$ pro druhou pozici a $x_2 = x_1, x_3 = x_2, x_1 < x_3$ pro třetí pozici. Z podmínek pro třetí pozici dostáváme spor, jelikož by muselo platit $x_1 < x_3 = x_2 = x_1$. Pro $k_{\max} = 2$ a $k_{\max} = 3$ by tedy byly výsledky totožné, protože se $b <_{\text{lex}}^3 a$ kvůli nalezenému sporu přeskočí. Případy $k_{\max} = 4$ a $k_{\max} = 5$ již nové podmínky také nepřinesou, protože nerovnost $b <_{\text{lex}}^4 a$ by zahrnovala podmínku $x_4 < x_4$ a podobně nerovnost $b <_{\text{lex}}^5 a$ by zahrnovala podmínku $x_5 < x_5$. Opět tedy v obou případech dostáváme spor. Jelikož jsme již prošli celý vektor, tak uvažovat vyšší $k_{\max}$ nedává smysl. Zadání vyšší hodnoty $k_{\max}$ však možné je a funkce v tomto případě jednoduše použije nejvyšší možnou hodnotu $k_{\max}$, která pro zadání dává smysl, což v tomto příkladu odpovídá $k_{\max} = 5$. To je zajištěno podmínkou $i = 1, \dots, \min(k_{\max}, n)$.

Regulace hloubky generování klauzulí je velmi důležitá. Zpravidla jsou totiž podmínky získané z $b <_{\text{lex}}^i a$ efektivnější pro menší i . Je to dáno především tím, že proměnné vyskytující se později ve vektoru proměnných mají nižší váhu a také část získaných podmínek může být už pokryta dříve vygenerovanými omezeními. Samotné generování klauzulí pro vyšší $k_{\max}$ také vede k delšímu času generování. Proto bychom mohli hledat „optimální“ $k_{\max}$, pro které rozbijeme dostatek symetrií, aby běh SAT řešiče byl dostatečně rychlý, ale pro které zároveň generování klauzulí netrvá příliš dlouho. Důležité je také poznamenat, že množina řešení je monotónně nerostoucí s rostoucím $k_{\max}$. To popíšeme následujícím lemmatem.

Lemma 4 (Monotónnost množiny řešení podle $k_{\max}$). *Nechť P je množina permutací, které odpovídají symetriím formule F . Označme $F_{P,k}$ formuli vzniklou z formule F přidáním klauzulí generovaných z permutací P do hloubky k , neboli přidáním podmínek získaných negací podmínek $\pi(x) <_{\text{lex}}^i x$ pro všechna $\pi \in P$ a všechna $i = 1, \dots, k$. Pokud $k_1 \leq k_2$, pak množina řešení formule F_{P,k_2} musí být obsažena v množině řešení formule F_{P,k_1} .*

Důkaz. Pokud máme $k_1 \leq k_2$, pak se v obou případech přidají podmínky získané negací podmínek $\bigvee_{\pi \in P} \bigvee_{i=1}^{k_1} \pi(x) <_{\text{lex}}^i x$. Pro k_2 pak navíc přibudou další podmínky pro $i = k_1 + 1, \dots, k_2$. Jelikož je tedy množina podmínek pro k_1 podmnožinou množiny podmínek pro k_2 , musí každé řešení formule F_{P,k_2} být zároveň řešením formule F_{P,k_1} . Žádná řešení tedy nemohou přibýt, mohou být pouze dalšími podmínkami odstraněna. $\square$

V každém kroku se uvažuje vzor odpovídající situaci, kdy jsou vektory a a b na všech předchozích pozicích stejné a na pozici i platí $b_i < a_i$. Jelikož pracujeme s binárními proměnnými, znamená tato ostrá nerovnost podmínky $b_i = 0$ a $a_i = 1$. Pokud je proměnná a_i rovna proměnné b_i , daná pozice nemůže být první pozicí, na které je b ostře menší než a , a proto se přeskočí.

Při konstrukci vzoru se dále udržuje množina aktivních proměnných. Aktivní proměnné jsou právě ty proměnné, které se v daném kroku skutečně podílejí na podmínkách určujících vzor. Jedná se tedy o proměnné vystupující v a a b na pozicích před i , kde požadujeme rovnost odpovídajících složek vektorů, a o proměnné a_i a b_i , které určují první ostrou nerovnost.

Rovnosti mezi proměnnými, které vznikají z podmínek $a_j = b_j$ pro $j < i$, a dále vztahy $a_i = 1, b_i = 0$, jsou ukládány pomocí datové struktury **Disjoint-Set**, známé také jako **Union-Find**. V implementaci je k tomu využita externí knihovna **DisjointSet**. Tato struktura umožňuje efektivně reprezentovat třídy ekvivalencí mezi proměnnými a speciálními reprezentanty hodnot 0 a 1. Díky tomu lze snadno zjistit, které proměnné musí mít v daném vzoru stejnou hodnotu, případně zda některá proměnná musí být rovna konstantě.

Po vytvoření všech rovností pro daný vzor se ověří, zda nevznikl spor mezi hodnotami 0 a 1. Pokud by se v jedné třídě ekvivalence ocitly současně reprezentanty obou těchto hodnot, znamenalo by to, že daný vzor není splnitelný. V takovém případě není potřeba pro tento vzor generovat žádnou klauzuli a algoritmus pokračuje dalším krokem.

Pro všechny aktivní proměnné se následně určí kanonický reprezentant jejich třídy ekvivalence. Tyto reprezentanty jsou v implementaci uloženy v množině *canon* a jsou určeny pomocí funkce z knihovny **DisjointSet**. Kanonické prvky slouží k tomu, aby bylo možné efektivně rozlišit, zda je daná proměnná ekvivalentní hodnotě 0, hodnotě 1, nebo jiné proměnné.

Pokud je proměnná v daném vzoru ekvivalentní hodnotě 0, přidá se do výsledné klauzule její kladný literál. Analogicky, pokud je proměnná ekvivalentní hodnotě 1, přidá se do klauzule její záporný literál. Výsledná klauzule tedy představuje negaci celého vzoru.

Příklad. Pro ujasnění si práci s výše popsanými prvky předvedeme na konkrétním příkladu. Uvažujme vektory proměnných

$$a = (x_1, x_2, x_3, x_4, x_5, x_6, x_7, x_8), b = (x_2, x_1, x_3, x_5, x_6, x_4, x_7, x_8).$$

Nechť jsme u pátého kroku, tedy $b <_{\text{lex}}^5 a$. Aktivní proměnné jsou zde ty proměnné, které se alespoň v jednom z vektorů vyskytují na pozicích 1,...,5. Jsou to tedy $x_1, x_2, x_3, x_4, x_5, x_6$. Z definice $b <_{\text{lex}}^5 a$ máme podmínky $x_6 < x_5, x_2 = x_1, x_1 = x_2, x_3 = x_3, x_5 = x_4$. Z rovností struktura **DisjointSet** vytvoří třídy

$$(x_1, x_2), (x_3), (x_4, x_5), (x_6).$$

Z ostré nerovnosti se určí $x_6 = 0$ a $x_5 = 1$, což se ve struktuře projeví jako

$$(x_1, x_2), (x_3), (x_4, x_5, \mathbf{1}), (x_6, \mathbf{0}).$$

Vidíme, že 0 a 1 nejsou ve stejné třídě, takže spor nedostáváme. Pro ostatní proměnné se vytvoří množina *canon*, kam se pro každou proměnnou uloží reprezentant z její třídy, který musí být pro danou třídu vždy stejný. Pro 0 a 1 je také

hledám reprezentant, kterým se ověřuje, jestli náleží do stejné třídy. Označme si tyto reprezentanty $c0$ a $c1$. V našem případě by toto mohlo vypadat například takto:

$$active = (x_1, x_2, x_3, x_4, x_5, x_6)$$

$$canon = (x_1, x_1, x_3, x_4, x_4, x_6), c0 = x_6, c1 = x_4$$

Následně už jen procházíme množinu *canon* a pro každou pozici nejdříve porovnáme, jestli je shodná s $c0$ nebo $c1$. Pokud není, tak danou pozici v *canon* srovnáme se stejnou pozicí v *active*. Podle toho určíme, jaké klauzule pro každou podmínku přidat. V našem případě to bude vypadat následovně.

1. Pro x_1 nepřidáváme žádné klauzule, protože odpovídající reprezentant v *canon* není roven $c0$ ani $c1$ a je to samotné x_1 (to se zjistí porovnáním s množinou *active*).
2. Pro x_2 opět není roven $c0$ ani $c1$, ale je to x_1 . Musíme tedy přidat klauzule pro $x_2 = x_1$ (reálně tedy $x_2 \neq x_1$, protože vše musíme negovat).
3. Příklad x_3 je stejný jako případ x_1 , takže nepřidáme žádné klauzule.
4. Pro x_4 i x_5 zjistíme, že jejich reprezentanty se rovnají $c1$, a tedy přidáváme klauzule pro $x_4 = 1$ a $x_5 = 1$.
5. Nakonec pro x_6 zjistíme, že reprezentant se rovná $c0$, takže $x_6 = 0$.

Zbývajícím případem je situace, kdy je proměnná ekvivalentní jiné proměnné, ale nikoliv přímo konstantě. V takovém případě je potřeba v negaci vzoru vyjádřit podmínku, že tyto dvě proměnné nemají stejnou hodnotu. Pro tuto nerovnost se zavádí pomocná proměnná reprezentující vztah $x \neq y$ pro proměnné x a y . Aby nebylo nutné pro stejnou dvojici proměnných vytvářet pomocnou proměnnou opakovaně, uchovává se jejich reprezentace ve struktuře *aux*.

Převod těchto pomocných podmínek do CNF tvaru je proveden pomocí Tseitinovy transformace [14]. Pro novou pomocnou proměnnou t , která reprezentuje vztah $x \neq y$, jsou přidány klauzule zajišťující ekvivalenci $t \leftrightarrow (x \neq y)$. Díky tomu lze v klauzuli použít pouze literál t , zatímco definice vztahu t a $x \neq y$ je zachycena samostatnými klauzulemi [14].

Po zpracování všech aktivních proměnných se výsledná klauzule přidá do množiny všech vygenerovaných klauzulí pro tuto dvojici vektorů proměnných. Tato klauzule zakazuje $b <_{\text{lex}}^i a$. Opakováním tohoto postupu pro všechny uvažované pozice i získáme klauzule, které zakazují, aby zpermutovaná varianta byla lexikograficky menší než původní řešení. Tím je dosaženo požadované podmínky $a \leq_{\text{lex}} b$, případně její omezené varianty určené parametrem k_{max} .

Algoritmus 3 Generování klauzulí z porovnání dvou vektorů proměnných

```
1: function GETCLAUSES( $a, b, k_{\max}, aux$ )
2:    $C \leftarrow \emptyset$ 
3:   for  $i = 0, \dots, \min(k_{\max} - 1, n - 1)$  do
4:     if  $a_i = b_i$  then
5:       continue
6:     end if
7:      $c \leftarrow \{(\neg a_i \vee b_i)\}$ 
8:     Zkonstruujeme všechny rovnosti a nerovnosti odpovídající vzoru  $pat_i$ 
9:     Přidáme podmínky  $a_i = 1$  a  $b_i = 0$ 
10:    Určíme třídy ekvivalencí proměnných
11:    if 0 náleží do stejné třídy ekvivalencí jako 1 then
12:      continue
13:    end if
14:    for all aktivní proměnné  $x$  do
15:      if  $x \equiv 0$  then
16:         $c \leftarrow c \cup \{x\}$ 
17:      else if  $x \equiv 1$  then
18:         $c \leftarrow c \cup \{\neg x\}$ 
19:      else if  $x \equiv y$  pro  $x \neq y$  then
20:        if  $\exists t \in aux$  takové, že  $t$  reprezentuje  $x \neq y$  then
21:           $c \leftarrow c \cup \{t\}$ 
22:        else
23:           $t :=$  pomocná proměnná reprezentující  $x \neq y$ 
24:           $C \leftarrow C \cup \{\text{klauzule definující reprezentaci pro } t\}$ 
25:           $c \leftarrow c \cup \{t\}$ 
26:        end if
27:      end if
28:    end for
29:     $C \leftarrow C \cup \{c\}$ 
30:  end for
31:  return ( $C, aux$ )
32: end function
```

2.5 Generování klauzulí pro rozbíjení symetrií

Tato funkce na vstupu dostává matici proměnných, typ symetrie, množinu permutací $perms$ a parametr $k_{\max}$. Množina $perms$ je vytvořena vybranou funkcí z funkcí popsanych v sekci 2.3. Podle typu symetrie funkce rozhodne, jakým způsobem budou tyto permutace aplikovány na binární matici reprezentující zadaný problém. Dále je v této funkci definována a udržována struktura pomocných proměnných aux .

Pro každou uvažovanou permutaci je pomocí funkce **GetPerm** vytvořena zpermutovaná varianta matice. Původní i zpermutovaná matice jsou následně převedeny na vektory a předány funkci **GetClauses**, která vygeneruje odpovídající klauzule. Ty jsou průběžně ukládány do výsledné množiny klauzulí.

Funkce **SymmetryClauses** tak tvoří zastřešující vrstvu celé implementace.

Zatímco samotná konstrukce klauzulí probíhá ve funkci **GetClauses**, úkolem Algoritmu 4 je zajistit správné generování a zpracování všech permutací odpovídajících zadané symetrii.

Algoritmus 4 Generování klauzulí pro rozbíjení symetrií

```

1: function SYMMETRYCLAUSES(matrix, symtype, perms,  $k_{max}$ )
2:   if  $k_{max} = 0$  then
3:     return  $\emptyset$ 
4:   end if
5:    $C \leftarrow \emptyset$ 
6:    $aux \leftarrow$  struktura pomocných proměnných
7:   for all  $(\pi_r, \pi_c) \in perms$  do
8:      $M' \leftarrow \text{GETPERM}(M, \pi_r, \pi_c)$ 
9:      $a \leftarrow \text{vec}(M)$ 
10:     $b \leftarrow \text{vec}(M')$ 
11:     $(C_\pi, aux) \leftarrow \text{GETCLAUSES}(a, b, k_{max}, aux)$ 
12:     $C \leftarrow C \cup C_\pi$ 
13:  end for
14:  return  $C$ 
15: end function

```

3 Experimentální část

V této kapitole popíšeme výsledky provedených experimentů. Nejprve si obecně popíšeme postup testování, způsob generování vstupních a výstupních souborů a sledované hodnoty. Následně se budeme věnovat jednotlivým skupinám grafů a jejich vyhodnocení.

Vstupní data byla vytvořena pomocí skriptu `geninput.py`, jehož fungování bylo popsáno v sekci 2.1. Celkem bylo vygenerováno 26 vstupních instancí odpovídajících problémům popsaným v sekci 1.3.

Generování výstupních SAT formulí probíhalo pomocí programu `sym.py`. Pro experimenty byla použita upravená verze tohoto programu, která umožňovala jednodušší hromadné spouštění programu na stejný vstup s různými parametry. Pro každý vstup byla použita určitá podmnožina množin permutací popsaných v sekci 2.3 a pro každou zvolenou množinu permutací bylo následně testováno několik hodnot $k_{\max}$.

Hodnoty $k_{\max}$ byly určeny podle celkového počtu proměnných v daném zadání. Pokud tento počet označíme K , nejprve jsme jej snížili na největší číslo K' takové, že $K' \leq K$ a K' je dělitelné pěti. Poté byly použity hodnoty

$$\frac{1}{5}K', \frac{2}{5}K', \frac{3}{5}K', \frac{4}{5}K', K'.$$

Kvůli jednotnému názvu výstupních souborů byla navíc použita také hodnota $k_{\max} = 0$. Program v tomto případě v podstatě jen vstup uložil do souboru s odlišným názvem pro jednodušší zpracování. Celkem tedy bylo pro každou zvolenou dvojici vstupu a množiny permutací uvažováno šest hodnot parametru $k_{\max}$.

Protože hodnota K' je ve většině případů menší než skutečný počet proměnných K , nejvyšší testovaná hodnota $k_{\max}$ neodpovídá úplnému průchodu celého vektoru proměnných. Nejsou tedy nutně vygenerovány všechny klauzule, které by bylo možné pro danou množinu permutací získat. Z předchozího testování však vyplynulo, že pro dostatečně velké instance bývají klauzule odpovídající posledním několika pozicím vektoru téměř vždy již pokryty dříve vygenerovanými omezeními a další symetrie zpravidla nerozsbíjejí. Toto zjednodušení je tedy pro uvažované instance prakticky přijatelné. U výrazně menších instancí by však takové omezení mohlo mít znatelnější vliv; například u formule s devíti proměnnými by ukončení generování již na hodnotě $k_{\max} = 5$ mohlo vynechat podstatnou část potenciálně užitečných klauzulí.

Kromě množin permutací implementovaných přímo v programu `sym.py` byla pro všechny problémy testována také množina označená jako `Satsuma`. Tato množina obsahuje generátory získané pomocí nástroje `satsuma` [5]. Pro její získání byla použita mírně upravená verze nástroje `satsuma`, která v rámci běhu uložila nalezenou množinu permutací do souboru. Takto získaná množina byla následně použita jako množina permutací pro `sym.py` namísto některé z množin permutací generovaných přímo naším nástrojem. Díky tomu můžeme porovnat, zda množina generátorů nalezená nástrojem `satsuma` vede k lepšímu rozbití symetrií než ostatní testované množiny generátorů. Samostatně byly navíc testovány také výstupní SAT formule vytvořené přímo nástrojem `satsuma`, který byl spuštěn ve variantě `satsuma lex ...`. Tento přístup je totiž bližší přístupu našeho nástroje. Druhou

možností je `satsuma fix ...`, která pracuje jinak a zdá se, že dosahuje obecně lepších výsledků. Výsledky první varianty nástroje `satsuma` budou porovnávány s výsledky našeho nástroje průběžně v dalších sekcích.

Všechny výsledné SAT formule byly testovány pomocí SAT řešiče `kissat` [15] a dále pomocí nástroje `d4` [16]. Nástroj `d4` umožňuje u splnitelných formulí spočítat celkový počet modelů. Tato hodnota je pro naše účely velmi důležitá, protože přímo ukazuje, kolik řešení zůstalo po přidání klauzulí pro rozbíjení symetrií. Na rozdíl od času běhu SAT řešiče, který může být ovlivněn mnoha dalšími faktory, je počet modelů mnohem lepším ukazatelem síly rozbití symetrií.

Počítání všech modelů je však výrazně náročnější než samotné rozhodnutí splnitelnosti. Z tohoto důvodu nebylo možné použít `d4` pro všechny testované instance. V mnoha případech výpočet nedoběhl v časovém limitu, který byl nastaven na 10 hodin. U těchto instancí proto sledujeme především čas běhu řešiče `kissat`. Pokud bylo možné použít `d4`, sledujeme také počet modelů výsledné formule.

U každého testu byly sledovány především tři hodnoty. První hodnotou je čas běhu našeho nástroje, tedy čas potřebný k vygenerování klauzulí pro rozbíjení symetrií. Tento čas ukládal program `sym.py` přímo do komentářů výstupní SAT formule. Druhou hodnotou je čas běhu externího nástroje, tedy buď `kissat`, nebo `d4`. Při porovnávání časů je důležité čas potřebný pro samotné vygenerování přidáných klauzulí brát v úvahu, protože pokud by zrychlení řešiče bylo menší než čas potřebný pro běh našeho nástroje, nemuselo by být použití nástroje pro danou instanci prakticky výhodné. Poslední hodnotou je počet modelů, pokud jej bylo možné spočítat.

Budeme porovnávat rozdíl mezi řešením původní formule a formule po přidání klauzulí pro rozbíjení symetrií, vliv různých hodnot parametru $k_{\max}$, vliv různých množin permutací a zejména rozdíly mezi generujícími a negenerujícími množinami. Zvláštní pozornost budeme věnovat také množině permutací získané z nástroje `satsuma` a porovnání výsledků našeho nástroje s výsledky nástroje `satsuma`.

Výsledky jsou v dalších částech uváděny v grafech vygenerovaných pomocným skriptem, který využíval knihovny `matplotlib` [17], `pandas` [18] a `numpy` [19]. Zpravidla jsou dvě varianty použitých grafů pro každý problém. Buď je použit graf s časy běhu řešiče `kissat` společně s grafem s časy běhu našeho nástroje, nebo je použit graf se součty časů běhu `d4` a našeho nástroje společně s grafem s počty různých modelů problému. V příloze je však pro každé zadání dostupné větší množství grafů obsahujících kromě časů a počtu řešení i poměr zrychlení oproti vstupu, poměr času běhu nástroje oproti `kissat` nebo `d4`, řadu heatmap a také souhrnné tabulky všech výsledků.

3.1 Problém holubníku

Výsledky pro problém holubníku jsou zobrazeny v grafech na obrázcích 3.1, 3.2, 3.3, 3.4 a 3.5. Všechny testované instance tohoto problému jsou nesplnitelné. Přestože bychom u splnitelných instancí mohli sledovat počet modelů, standardně se u tohoto problému uvažují jen nesplnitelné instance a navíc splnitelných instancí uvidíme dostatek u ostatních problémech.

Ze všech námi vytvořených testovaných množin permutací dosahuje jednoznačně nejlepších výsledků množina `Neighbors`. U všech testovaných instancí zde

dochází k výraznému zkrácení času běhu SAT řešiče, přičemž čas potřebný pro samotné vygenerování klauzulí se pohybuje nejvýše okolo jedné sekundy. Získané zrychlení je tedy řádově vyšší než režie spojená s během našeho nástroje, takže jeho použití je v tomto případě jednoznačně přínosné.

Naopak většina ostatních testovaných množin permutací nedokáže běh SAT řešiče zrychlit natolik, aby byly náročnější instance vyřešeny před uplynutím nastaveného časového limitu. Pravděpodobným důvodem je, že nesplnitelné instance problému holubníku se stávají pro SAT řešič velmi obtížnými již při relativně malém počtu symetrických holubů a holubníků. Pokud po přidání klauzulí pro rozbíjení symetrií zůstane dostatečně velká symetrická podmnožina řádků nebo sloupců, zůstává problém výpočetně velmi náročný.

Množina **Neighbors** se od ostatních množin liší tím, že obsahuje transpozice odpovídající záměnám všech sousedních řádků i všech sousedních sloupců matice. Rozbíjení symetrií je tak rovnoměrně rozprostřeno přes celý vektor proměnných a nezůstávají větší souvislé oblasti se zachovanou symetrií. Naproti tomu množiny **Randtrans** a **Nongen** sice obsahují stejný počet permutací jako **Neighbors**, jejich permutace jsou však vybírány náhodně, takže není zaručeno jejich rovnoměrné rozmístění mezi všemi proměnnými. Přesto právě tyto dvě množiny dosahují po množině **Neighbors** nejlepších výsledků, což naznačuje, že samotný počet použitých permutací hraje významnou roli.

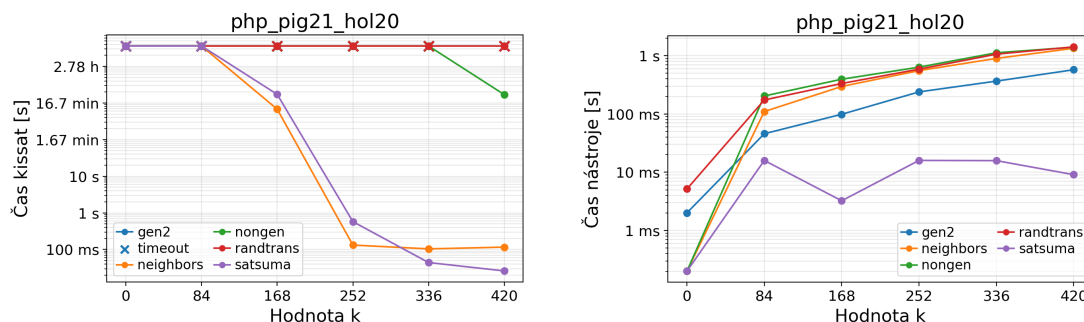
Množina **Transpositions** se pro uvažované velikosti instancí ukázala jako prakticky nepoužitelná, protože pro rozměry binárních matic uvažované v uvedených problémech byl nárůst času potřebného ke generování klauzulí tak vysoký, že buď ve většině případů běh nástroje ani neskončil v rozumném čase, nebo by oproti množině **Neighbors** byl velmi nevýhodný. Pro problém holubníku uvažujeme množinu **Transpositions** zvlášť pro řádky a pro sloupce a poté procházíme jejich kartézský součin postupem popsáním v sekci 1.2. To znamená, že pro *pig* řádků máme celkem $\frac{pig(pig-1)}{2}$ permutací a pro *hol* sloupců celkem $\frac{hol(hol-1)}{2}$ permutací. Ke každé množině přidáme identitu a poté procházíme všechny možné dvojice prvků z těchto množin (kromě dvojice, ve které je na obou pozicích příslušná identická permutace). Celkem dostáváme $(\frac{pig(pig-1)}{2} + 1)(\frac{hol(hol-1)}{2} + 1) - 1$ různých permutovaných vektorů proměnných. Například pro **php_pig31_hol30** dostáváme tedy $(\frac{31(31-1)}{2} + 1)(\frac{30(30-1)}{2} + 1) - 1 = 203175$. Množina **Neighbors** v tomto případě pro řádky obsahuje 30 permutací a pro sloupce 29 permutací. Analogickým postupem se dostaneme k celkovému počtu 929 různých permutovaných vektorů proměnných. Jak je vidět, rozdíl mezi počty generovaných permutovaných vektorů proměnných je více než dvouseťnásobný. Není proto překvapivé, že se tato skutečnost výrazně projeví na době běhu našeho nástroje. Tento rozdíl navíc s rostoucí velikostí problému dále narůstá, protože počet permutací v množině **Transpositions** roste kvadraticky v počtu řádků i sloupců ($\Theta(pig^2 \cdot hol^2)$), zatímco u množiny **Neighbors** pouze lineárně ($\Theta(pig \cdot hol)$).

Na instanci **php_pig40_hol10** lze pozorovat zajímavý jev, který se bude opakovat i u dalších problémů. Nejkratší čas běhu SAT řešiče zde není dosažen pro nejvyšší hodnotu k_{max} , ale pro některou z nižších hodnot. Přestože vyšší hodnoty k_{max} vždy rozbíjí alespoň tolik symetrií jako nižší hodnoty, zároveň vedou k vytvoření většího množství pomocných proměnných a klauzulí. V některých případech může být režie spojená s jejich zpracováním větší než přínos více rozbitých symetrií. Při hledání vhodné hodnoty parametru k_{max} je proto nutné zohlednit nejen

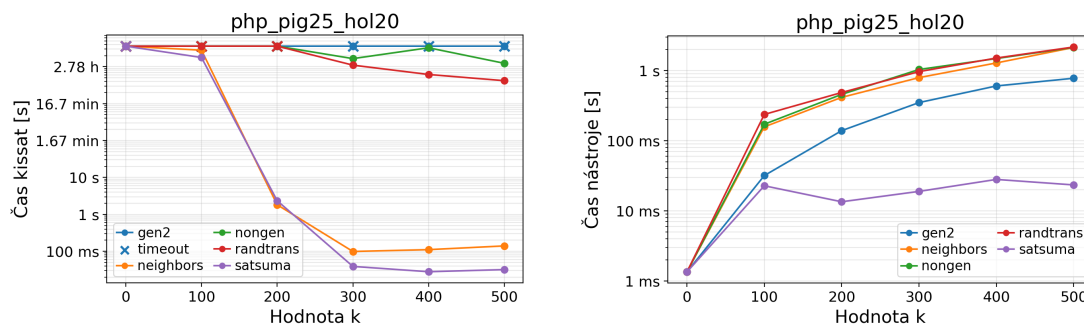
účinnost rozbití symetrií a čas generování klauzulí, ale také vliv vyššího množství proměnných na výkon SAT řešiče.

Kromě **Neighbors** je jedinou výrazně účinnější množinou permutací množina **Satsuma**, která **Neighbors** dokonce ve většině případů překonává. Co je na tom nejvíce překvapivé je to, že množina **Satsuma** obsahuje výrazně méně permutací než množina **Neighbors**. Při předchozím testování se totiž ukázalo, že celkový počet použitých permutací má obecně významný vliv na účinnost rozbití symetrií. Zde se celkový počet permutací v množině **Satsuma** pro testované instance pohybuje v rozmezí od 40 do 59, zatímco v případě **Neighbors** se používá kartézský součin transpozic pro řádky a pro sloupce. To znamená celkem $pig \cdot hol - 1$ permutací, což se zde pohybuje v řádu několika stovek pro všechny instance. Permutace nalezené při detekci symetrií nástrojem **satsuma** tedy v tomto případě opravdu dávají velmi dobré částečné rozbití symetrie, jak bylo zmíněno v první kapitole. Jedním z možných vysvětlení je skutečnost, že problém holubníku patří mezi nejčastěji používané benchmarky při výzkumu rozbíjení symetrií, a i nástroje pro detekci symetrií proto mohou být na podobné struktury obzvláště dobře přizpůsobeny.

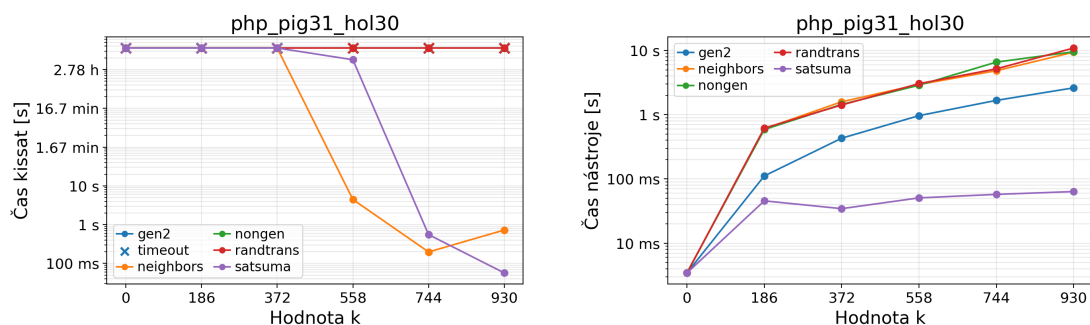
Nakonec pokud srovnáme nástroj **satsuma** a náš nástroj s množinou **Satsuma**, dostaneme téměř totožné výsledky. SAT řešič v obou případech zjistí téměř okamžitě, že je problém nespílitelný.



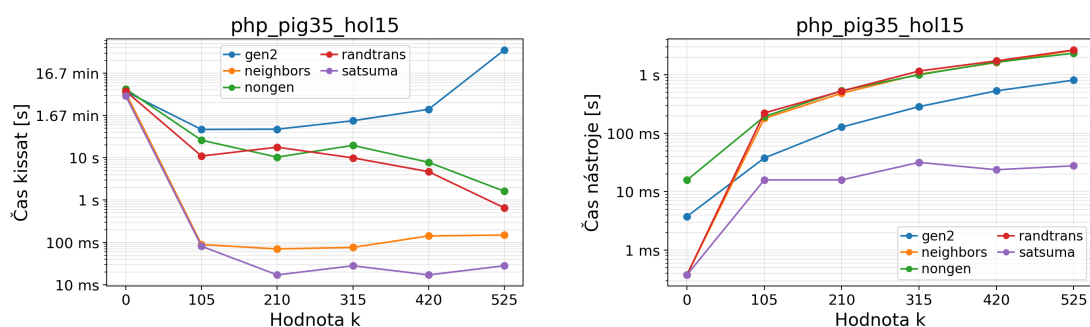
Obrázek 3.1 Čas kissat a čas nástroje pro `php_pig21_hol20`



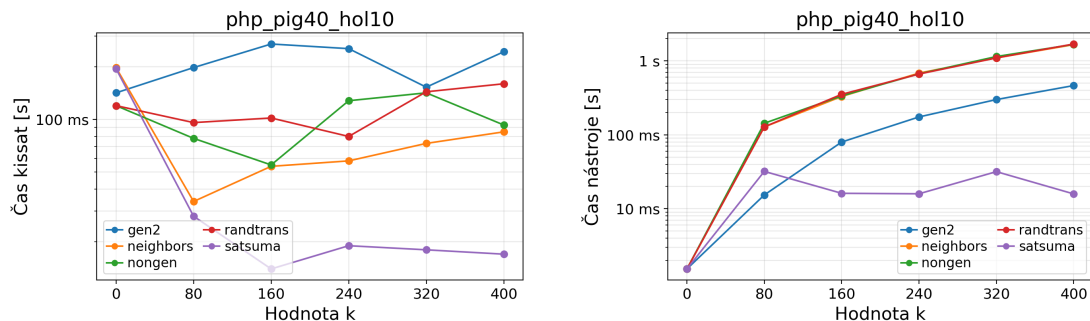
Obrázek 3.2 Čas kissat a čas nástroje pro `php_pig25_hol20`



Obrázek 3.3 Čas kissat a čas nástroje pro `php_pig31_hol30`



Obrázek 3.4 Čas kissat a čas nástroje pro `php_pig35_hol15`



Obrázek 3.5 Čas kissat a čas nástroje pro `php_pig40_hol10`

3.2 Problém N královen

Výsledky pro problém N královen jsou zobrazeny na obrázcích 3.6, 3.7, 3.8, 3.9 a 3.10. Na rozdíl od problému holubníku jsou všechny testované instance splnitelné. Přestože nalezení jednoho řešení je díky vysokému počtu různých řešení pro SAT řešič poměrně jednoduché, je zajímavé sledovat právě celkový počet modelů nalezený nástrojem `d4`.

Pro tento problém porovnáváme pouze naši generující množinu permutací dihedrální grupy `DihedralGen` a množinu `Satsuma`. Množina `DihedralGen` obsahuje vždy dvě permutace, zatímco množina `Satsuma` obsahuje dvě nebo tři permutace. Množina obsahuje tři permutace v případech $n = 10$ a $n = 13$. Právě v těchto případech můžeme v grafech pozorovat nejvýraznější rozdíly mezi testovanými

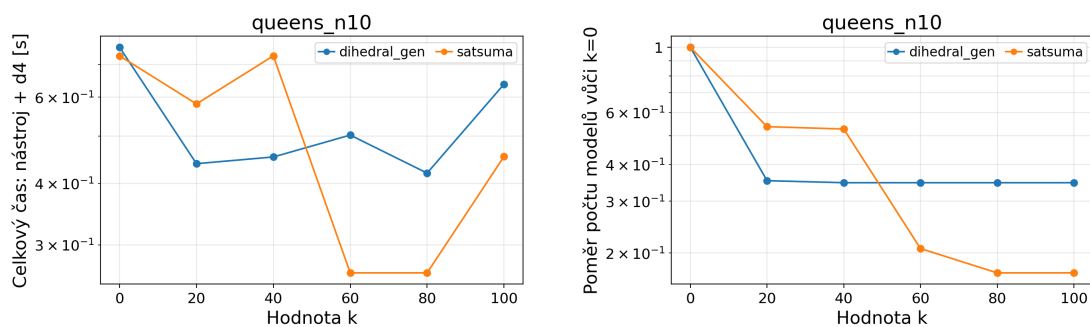
množinami permutací. U ostatních instancí, kde jsou množiny stejně velké, již rozdíly nejsou tak výrazné. Toto pozorování potvrzuje významný vliv počtu použitých permutací na účinnost rozbíjení symetrií, který byl zmíněn v předchozí sekci.

Problém N královen se od ostatních problémů liší také tím, že počet jeho symetrií nezávisí na velikosti instance. Pro všechna n jsou symetrie dány osmi symetriemi čtvercové šachovnice tvořícími dihedralní grupu. Se zvyšující se velikostí šachovnice se tedy zvětšuje pouze SAT formule, zatímco množina symetrií zůstává stále stejná.

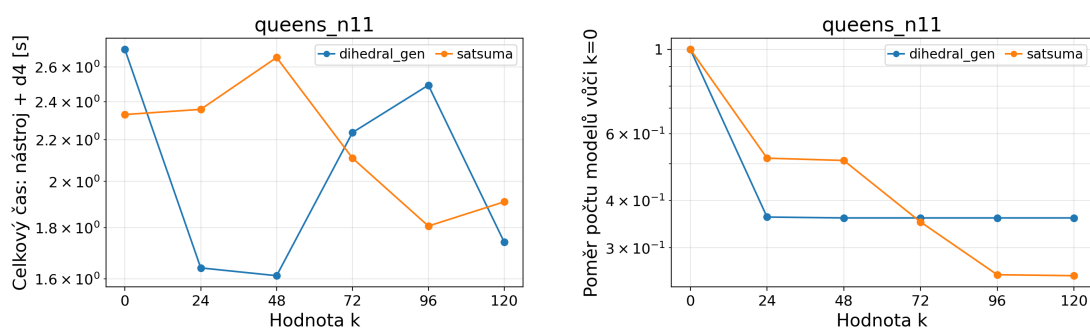
V porovnání s nástrojem **satsuma** jsou výsledky našeho nástroje podobné. Výraznější rozdíly lze pozorovat pouze u instancí, u kterých nástroj **satsuma** při vlastním běhu našel pravděpodobně množinu o třech permutacích, zatímco při exportu množiny byly nalezeny pouze dvě permutace. Tyto dva běhy totiž nebyly prováděny současně a nástroj může nalézt pro totožné zadání různé množiny generátorů pro různé běhy.

Časy běhu nástroje **d4** jsou u většiny testovaných instancí velmi malé, a proto z nich nelze vyvozovat jednoznačné závěry. Obecně nejvíce vypovídajícím se jeví případ s nejvyšším n , tedy $n = 14$. Zde už se časy běhu pohybují v řádu minut a lze jednoznačně vidět, že pro množinu **DihedralGen** se čas významně snížil pro první nenulovou hodnotu k_{max} a poté postupně roste. Toto chování odpovídá také grafům s počty modelů. Dramatický pokles totiž nastane právě pro první nenulovou hodnotu k_{max} a poté už klesá jen velmi mírně. Tím pádem pro vyšší k_{max} už téměř žádné zrychlení rozbíjení symetrií nepřináší a přidávání dalších pomocných proměnných jen řešení zpomaluje. Proto čas roste.

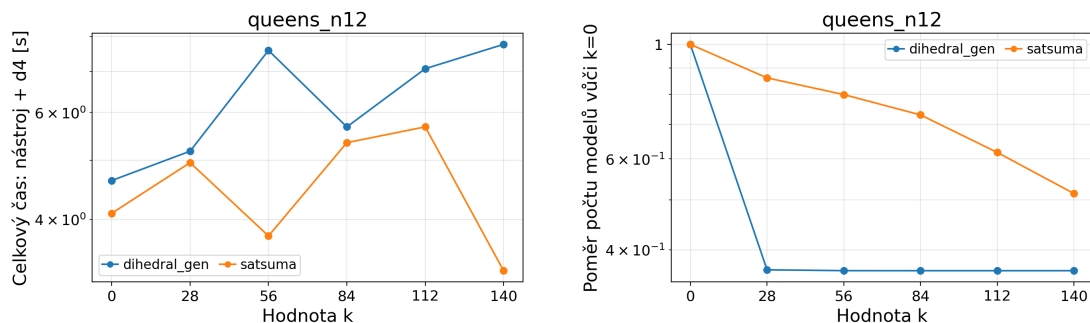
Zajímavou vlastností grafů s počtem modelů je jejich konzistence napříč různými velikostmi problému. Pro množinu **DihedralGen**, která obsahuje vždy stejné dva generátory dihedralní grupy, se poměr počtu modelů po rozbití symetrií vůči původnímu počtu modelů ve všech testovaných instancích pohybuje lehce pod hodnotou $\frac{4}{10}$. Stejně konzistentně se projevuje i závislost na parametru k_{max} , kdy největší pokles počtu modelů nastává již pro první nenulovou hodnotu tohoto parametru. U množiny **Satsuma** lze pozorovat, že výraznější poklesy počtu modelů nastávají v různých instancích při různých hodnotách k_{max} , což naznačuje, že optimální hloubka generování by mohla záviset na konkrétních použitých permutacích. U množiny **Satsuma** se totiž narozdíl od množiny **DihedralGen** mohou pro různé běhy vyskytovat různé generátory dihedralní grupy. Při dalším testování by tedy bylo žádoucí zahrnout všech osm permutací a sledovat, pro které hodnoty k_{max} se jednotlivé permutace projeví nejvýrazněji a jestli jsou hodnoty podobně konzistentní napříč různými velikostmi zadání.



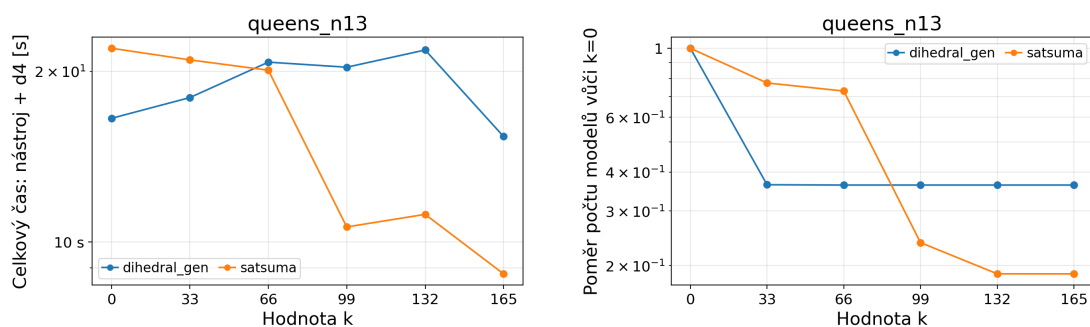
Obrázek 3.6 Čas nástroje + čas d4 a počet modelů pro queens_n10



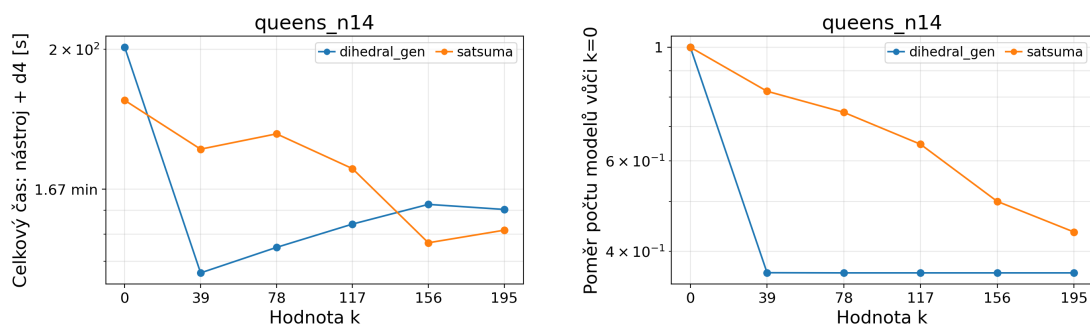
Obrázek 3.7 Čas nástroje + čas d4 a počet modelů pro queens_n11



Obrázek 3.8 Čas nástroje + čas d4 a počet modelů pro queens_n12



Obrázek 3.9 Čas nástroje + čas d4 a počet modelů pro queens_n13



Obrázek 3.10 Čas nástroje + čas d4 a počet modelů pro queens_n14

3.3 Problém nalezení matice s řádky a sloupci se zadanými součty

Výsledky experimentů pro problém nalezení matice s řádky a sloupci se zadanými součty jsou uvedeny na obrázcích 3.11, 3.12, 3.13 a 3.14. Testovány byly také instance s parametry $rc = 10, rcrand = 5$ a $rc = 13, rcrand = 10$. Jelikož však jejich výsledky byly velmi podobné instanci s parametry $rc = 15, rcrand = 3$, nejsou z důvodu úspory místa v hlavním textu uvedeny. Jejich grafy jsou dostupné v příloze práce.

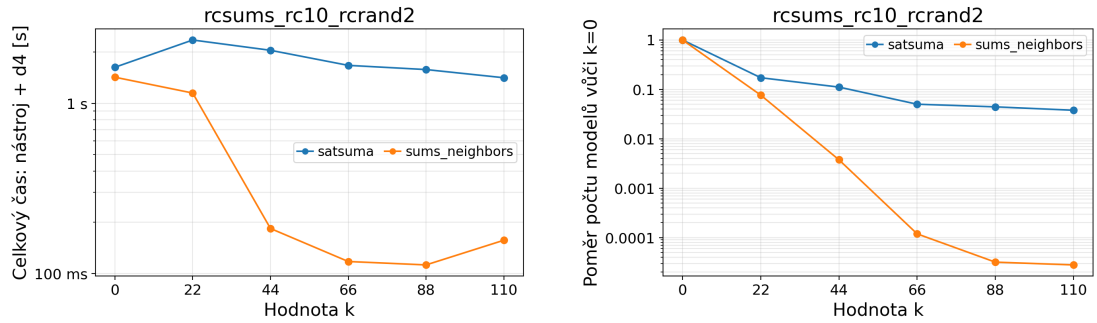
Porovnávat v tomto případě můžeme opět pouze dvě množiny permutací a to množinu **SumsNeighbors** a množinu **Satsuma**. U instancí na obrázcích 3.11 a 3.12 lze pozorovat velmi podobné chování jako u množiny **Neighbors** v problému holubníku. Dochází zde k výraznému snížení počtu modelů i času běhu řešiče a účinnost rozbíjení symetrií se stále zvyšuje i pro vyšší hodnoty parametru k_{max} . Podobnost není náhodná. Pokud uvažujeme matici o 11 řádcích a 10 sloupcích, ve které mají všechny řádky i všechny sloupce předepsaný součet roven jedné, dostáváme téměř stejný problém jako je problém holubníku s 11 holuby a 10 holubníky.

U ostatních testovaných instancí již experimenty neposkytují tak jednoznačné závěry. Ve splnitelných instancích je počet modelů natolik vysoký, že nástroj **d4** nedokončí jejich spočítání v daném časovém limitu, zatímco SAT řešič **kissat** nalezne jedno řešení téměř okamžitě. Naměřené časy řešení se proto pohybují v řádu milisekund a jejich vzájemné rozdíly jsou velmi malé. Tyto rozdíly mohou být ovlivněny řadou dalších faktorů uvnitř SAT řešiče a nelze z nich proto vyvozovat významnější závěry o kvalitě jednotlivých množin permutací. Obdobná situace nastává u nesplnitelné instance na obrázku 3.13, kde je pravděpodobné, že tato instance obsahuje poměrně jednoduchý spor vyplývající ze zvolených hodnot parametrů rc a $rcrand$, takže SAT řešič dokáže nesplnitelnost prokázat téměř okamžitě a opět čas ovlivňují spíše náhodné vlivy běhu SAT řešiče.

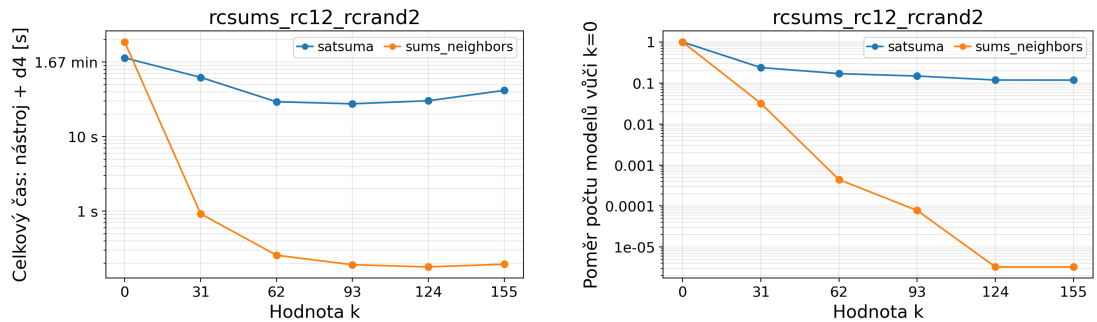
Při porovnání množiny **Satsuma** a samotného nástroje **satsuma** dostáváme pro tento problém odlišné výsledky než u předchozích experimentů. Množina **Satsuma** obsahuje podstatně méně permutací než množina **SumsNeighbors**, což se projevuje nižší účinností rozbíjení symetrií. Ani samotný nástroj **satsuma** zde nedosahuje výsledků srovnatelných s naším nástrojem využívajícím množinu **SumsNeighbors**. U instancí na obrázcích 3.11 a 3.12 jsou výsledky nástroje **satsuma** pouze mírně lepší než výsledky našeho nástroje používajícího exportovanou množinu **Satsuma**,

zatímco množina `SumsNeighbors` vede k výrazně lepším výsledkům než oba tyto přístupy.

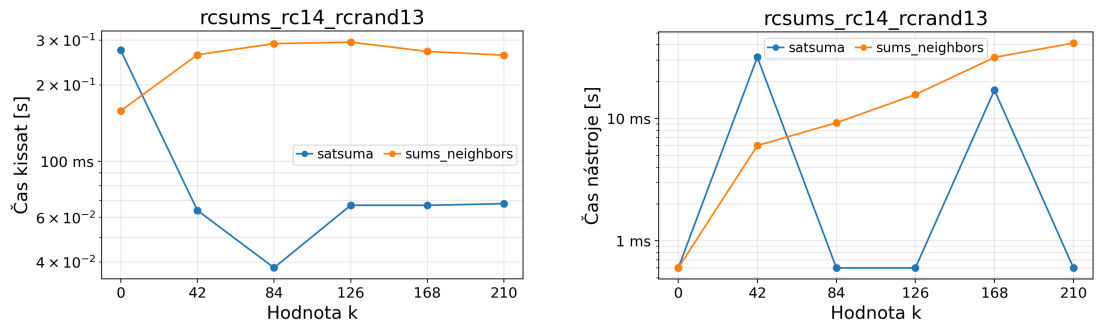
Možným vysvětlením je charakter symetrií tohoto problému. Množina permutací `SumsNeighbors` je navržena přímo pro symetrie vznikající mezi řádky nebo sloupci se stejnými součty, zatímco nástroj `satsuma` detekuje symetrie pouze ze zadané SAT formule. Zdá se tedy, že symetrie pouze na podmnožinách řádků nebo sloupců jsou pro nástroj `satsuma` těžko detekovatelné.



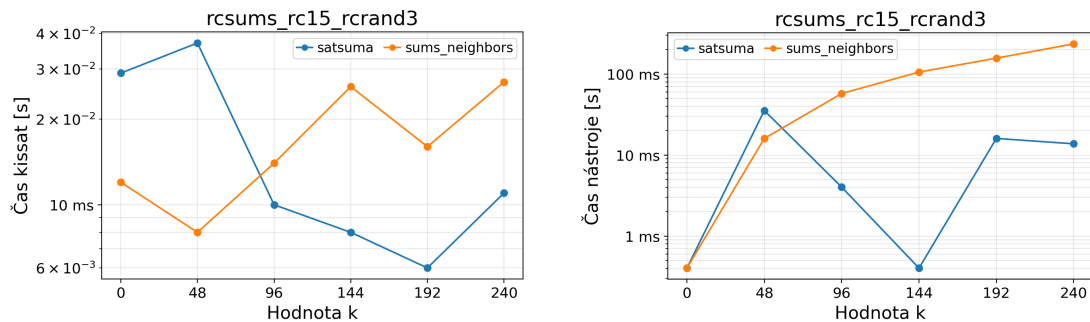
Obrázek 3.11 Čas nástroje + čas d4 a počet modelů pro `rcsums_rc10_rcrand2`



Obrázek 3.12 Čas nástroje + čas d4 a počet modelů pro `rcsums_rc12_rcrand2`



Obrázek 3.13 Čas kissat a čas nástroje pro `rcsums_rc14_rcrand13`



Obrázek 3.14 Čas kissat a čas nástroje pro rcsums_rc15_rcrand3

3.4 Problém latinských čtverců

Výsledky pro problém latinských čtverců jsou uvedeny na obrázcích 3.15, 3.16, 3.17 a 3.18. Jedná se o první testovaný problém o třech rozměrech, kterými jsou řádky, sloupce a hodnoty symbolů, které do matice rozmisťujeme. Z důvodu úspory místa nejsou uvedeny výsledky pro instanci $lsq = 9$, které jsou velmi podobné instanci $lsq = 8$. Grafy pro tuto instanci jsou opět uvedeny v příloze práce.

U dvou nejmenších instancí lze porovnat dosud největší počet různých množin permutací. Nejprve se zaměříme na množiny **Neighbors**, **Randtrans** a **Nongen**. Rozdíly u těchto množin jsme mohli zatím sledovat pouze omezeně u problému holubníku a chceme na nich pozorovat, zda skutečnost, že množina permutací tvoří generující množinu grupy symetrií, vede také k efektivnějšímu rozbíjení symetrií. Ve všech testovaných případech dosahuje opět zdaleka nejlepších výsledků množina **Neighbors**. Ta využívá transpozice sousedních prvků rovnoměrně rozložené přes všechny tři rozměry problému. Naproti tomu množiny **Randtrans** a **Nongen** využívají náhodně vybrané transpozice. Stejně jako u problému holubníku se tak potvrzuje výhoda rovnoměrného pokrytí proměnných.

Zajímavější je vzájemné porovnání množin **Randtrans** a **Nongen**. Obě množiny obsahují stejný počet permutací a liší se především tím, že **Randtrans** může tvořit generující množinu příslušné grupy symetrií, zatímco **Nongen** nikoliv. Ve všech testovaných případech kromě jediné výjimky pro $lsq = 5$ a $k_{max} = 24$ dosahuje množina **Randtrans** lepších výsledků jak z hlediska počtu modelů, tak i času běhu řešiče. Stejný závěr bylo možné pozorovat také v několika předchozích sériích experimentů. Výsledky proto naznačují, že vlastnost být generující množinou může mít pozitivní vliv na účinnost rozbíjení symetrií. Vyplývá z toho, že by mohlo mít smysl v budoucnu tuto vlastnost studovat i teoreticky a pokusit se ji dokázat, nebo případně vyvrátit, pokud na výsledky testů mají vliv jiné nyní neznámé faktory.

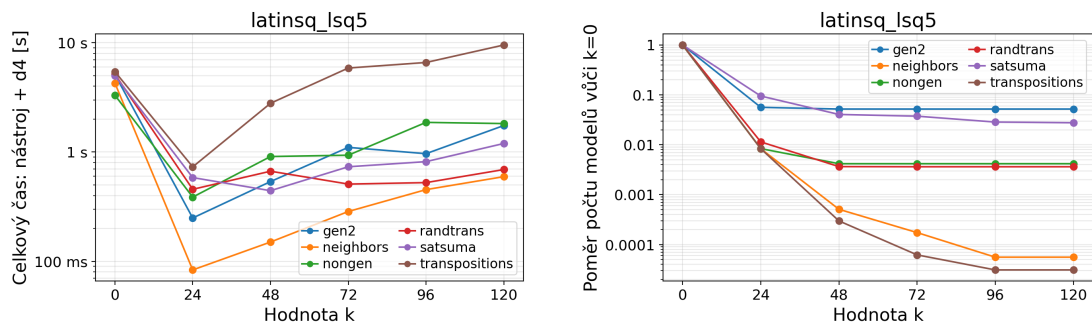
Další zajímavé srovnání představují množiny **Gen2** a **Satsuma**. Množina permutací **Satsuma** obsahuje ve všech testovaných instancích pět permutací, zatímco množina **Gen2** pouze dvě. Samotný počet permutací však v tomto případě nevystihuje skutečný počet použitých permutovaných vektorů proměnných. Permutace množiny **Satsuma** jsou definovány přímo na celém vektoru proměnných, zatímco permutace množiny **Gen2** jsou definovány samostatně pro řádky, sloupce a hodnoty symbolů. Náš nástroj proto vytváří všechny jejich možné kombinace. Trojice mohou na všech pozicích kromě permutací z **Gen2** také obsahovat identickou

permutaci. Celkem tak využívá $3^3 - 1 = 26$ různých permutací, přičemž jedinou vynechanou kombinací je trojice identických permutací na všech třech pozicích. Skutečný počet použitých permutací je tedy i v tomto případě vyšší než u množiny **Satsuma**, což se odráží také na výsledcích.

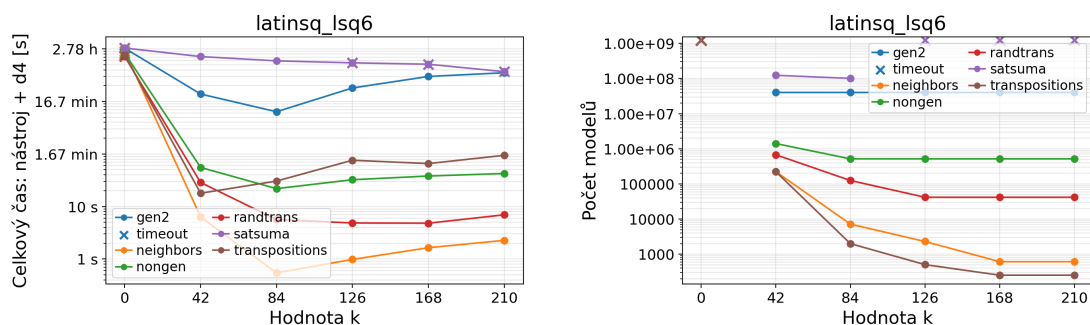
Nástroj **satsuma** i v tomto případě dosahuje velmi podobných výsledků jako náš nástroj využívající množinu **Satsuma**. To naznačuje, že navržený způsob generování Lex-Leader klauzulí dokáže nalezenou množinu generátorů využít srovnatelně efektivně jako nástroj **satsuma**. Výsledky v grafech pro množinu **Satsuma** tedy můžeme zároveň považovat za výsledky nástroje **satsuma**. Ve srovnání s množinou **Neighbors** však náš nástroj dosahuje lepších výsledků z hlediska redukce počtu modelů.

Za zmínku stojí také množina **Transpositions**. Přestože její použití vede kvůli velkému počtu permutací k výrazně delším časům generování i řešení, z hlediska samotného počtu odstraněných symetrií dosahuje ještě lepších výsledků než množina **Neighbors**. Počet modelů se zde snižuje až na hodnoty 5, respektive 252, pro instance $lsq = 5$, respektive $lsq = 6$. Zároveň je to jeden z mála případů, kdy se na celkovém čase významněji podílí čas běhu nástroje. Proto množinu **Transpositions** nelze použít pro $lsq > 6$. Již pro $lsq = 7$ nebylo možné vygenerovat klauzule pro všechny používané hodnoty k_{max} . Navíc se kromě času dramaticky zvyšovala i velikost výsledného souboru. To bylo mimochodem limitujícím faktorem i pro řadu dalších instancí u všech uvažovaných problémů.

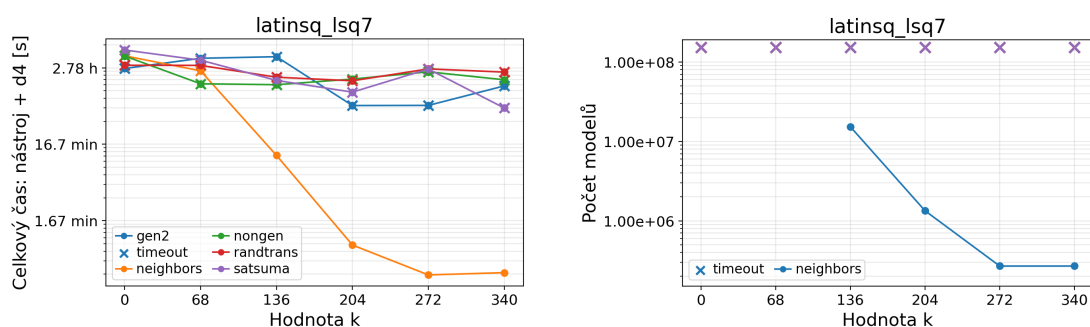
Pro $lsq = 8$ (a $lsq = 9$) nastane již známá situace, kdy počet řešení je tak velký, že běh nástroje **d4** ani po přidání klauzulí pro rozbíjení symetrií neskončí v časovém limitu, ale naopak nástroj **kissat** díky velkému počtu řešení nalezne jedno řešení téměř okamžitě. Proto časy kolísají a spíše se zhoršují s přidáváním většího množství pomocných proměnných. Díky nižšímu počtu permutací zde pro množinu **Satsuma** čas stoupá pomaleji, protože není přidáváno takové množství pomocných proměnných.



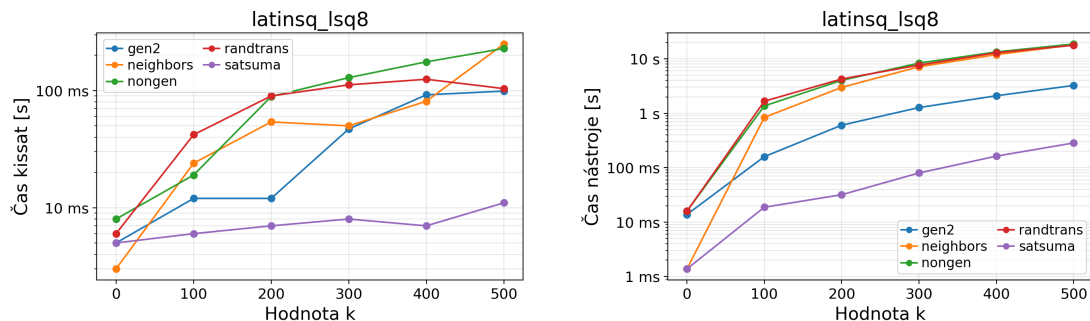
Obrázek 3.15 Čas nástroje + čas d4 a počet modelů pro golf_g3_p3_w3



Obrázek 3.16 Čas nástroje + čas d4 a počet modelů pro latinsq_ls6



Obrázek 3.17 Čas nástroje + čas d4 a počet modelů pro latinsq_ls7



Obrázek 3.18 Čas kissat a čas nástroje pro latinsq_ls8

3.5 Problém společenských golfistů

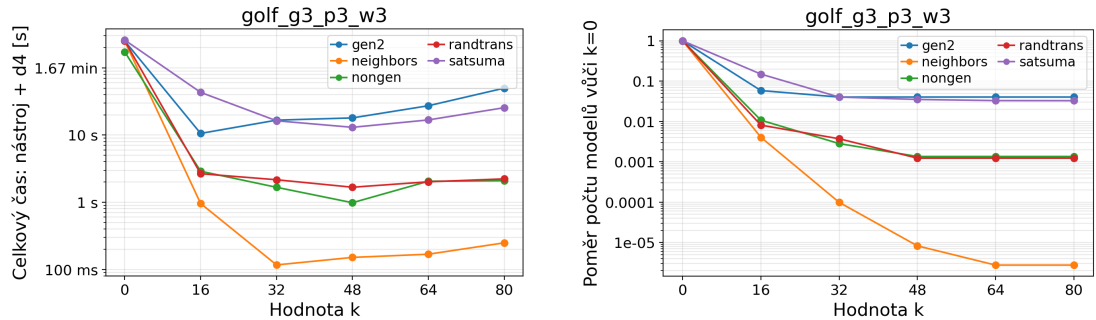
Výsledky pro problém společenských golfistů jsou uvedeny na obrázcích 3.19, 3.20 a 3.21. Pro přehlednost nejsou zobrazeny grafy pro instance `golf_g5_p4_w4` a `golf_g6_p2_w3`, které jsou stejně jako u předchozích problémů uvedeny v příloze práce. Jejich výsledky jsou velmi podobné instanci `golf_g5_p3_w6`. Tyto instance mají podobně jako už několikrát u předchozích problémů příliš mnoho řešení a tím pádem nástroj `d4` neskončí v časovém limitu, ale nástroj `kissat` z velkého množství řešení najde jedno velmi rychle.

Zajímavější jsou proto výsledky na obrázcích 3.19 a 3.20, kde jedna z testovaných instancí je splnitelná a druhá nespílitelná. Také zde dosahuje nejlepších výsledků množina `Neighbors`, což je v souladu s výsledky všech předchozích

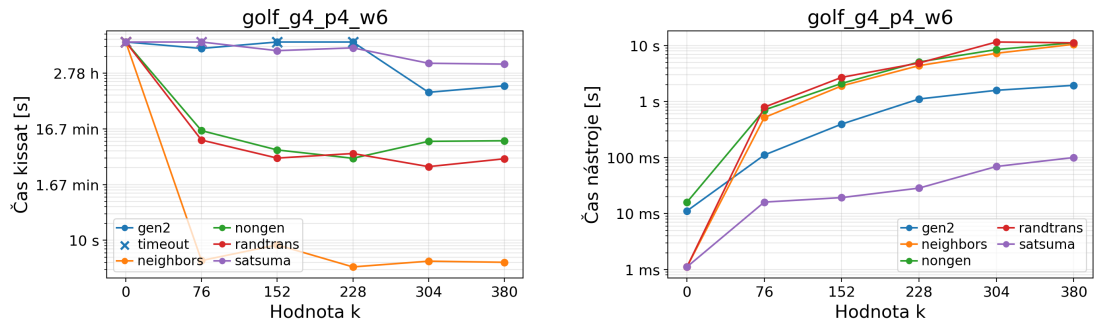
experimentů. Zajímavé je porovnání množin **Randtrans** a **Nongen**. V několika případech pro nižší hodnoty parametru k_{max} dosahuje množina **Nongen** mírně lepších výsledků než **Randtrans**. Přestože jsou obě množiny permutací voleny náhodně, takže bychom mohli v krajních případech očekávat občas i výrazné vítězství množiny **Nongen**, nastává podobná situace velmi ojediněle. Pro vyšší hodnoty k_{max} již množina **Randtrans** opět dosahuje lepších výsledků, takže i tento experiment podporuje hypotézu, že generující množiny permutací mohou být pro rozbíjení symetrií obecně vhodnější než stejně velké negenerující množiny.

Podobně jako u problému latinských čtverců lze porovnat také množiny **Gen2** a **Satsuma**, kdy množina **Gen2** opět odpovídá celkem 26 použitým permutacím, zatímco množina **Satsuma** obsahuje 18 permutací pro instanci **golf_g3_p3_w3** a 13 permutací pro instanci **golf_g4_p4_w6**. Přesto jsou výsledky obou množin velmi podobné. To naznačuje, že množiny generátorů nalezené nástrojem **satsuma** při detekci symetrií mají vyšší kvalitu a dokážou i při menším počtu permutací rozbít symetrie účinně.

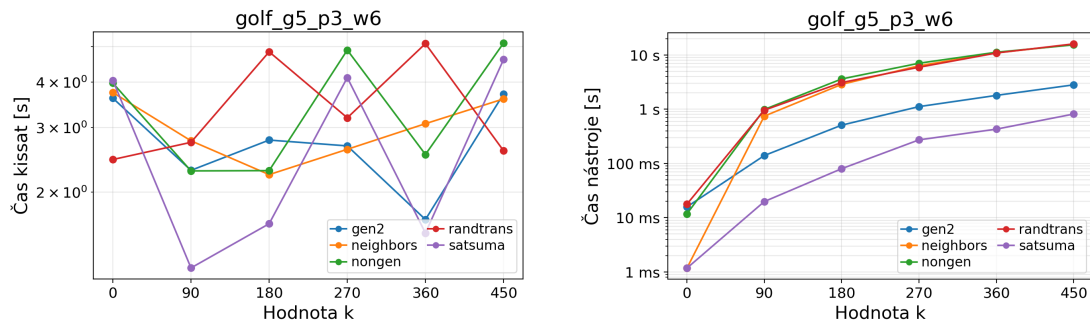
Pro nástroj **satsuma** je například u **golf_g3_p3_w3** čas i počet modelů zhruba poloviční oproti našemu nástroji s množinou **Satsuma**, což ale může být způsobeno rozdílnou nalezenou množinou permutací. Počet modelů po přidání klauzulí je zde 44106. Pro srovnání náš nástroj s množinou **Neighbors** se dostal na 12 modelů. Stejně tak u nesplnitelné instance **golf_g4_p4_w6** jsou početnější množiny výrazně účinnější.



Obrázek 3.19 Čas nástroje + čas d4 a počet modelů pro **golf_g3_p3_w3**



Obrázek 3.20 Čas kissat a čas nástroje pro **golf_g4_p4_w6**



Obrázek 3.21 Čas kissat a čas nástroje pro golf_g5_p3_w6

3.6 Shrnutí experimentálních výsledků

Provedené experimenty ukázaly, že navržený nástroj je schopen významně urychlit řešení řady SAT formulí pomocí rozbíjení symetrií. Největší přínos lze pozorovat především u obtížných nesplnitelných instancí, kde redukce symetrických řešení vede k výraznému zkrácení času běhu SAT řešiče. Naopak u úloh s velmi vysokým počtem řešení bývá nalezení jednoho modelu i bez rozbíjení symetrií velmi rychlé, a proto se zrychlení běhu SAT řešiče projeví jen omezeně. V těchto případech se jako vhodnější ukázalo sledování celkového počtu modelů, který lépe vystihuje účinnost rozbití symetrií.

Ve všech testovaných problémech se jako nejefektivnější z implementovaných množin permutací ukázala množina **Neighbors**, pokud samozřejmě je na daný problém aplikovatelná. Tato množina rovnoměrně pokrývá celý prostor proměnných pomocí transpozic sousedních prvků, což se opakovaně ukázalo jako velmi účinný způsob rozbíjení symetrií. Navíc je v porovnání s množinou **Transpositions** kompromisem mezi velikostí množiny a účinností rozbíjení symetrií. Množina **Transpositions** sice je schopna rozbít více symetrií, ale kvůli své velikosti je pro většinu zadání prakticky nepoužitelná.

Dalším opakovaným pozorováním bylo, že generující množiny permutací dosahovaly téměř ve všech experimentech lepších výsledků než stejně velké negenerující množiny. Tento trend se projevil při porovnání množin **Randtrans** a **Nongen**. Přestože experimentální výsledky samy o sobě nemohou tuto vlastnost dokázat, naznačují, že skutečnost, že množina permutací generuje celou grupu symetrií, může mít pozitivní vliv na účinnost rozbíjení symetrií a mělo by smysl se tímto v budoucnu teoreticky zabývat.

Významný vliv na výsledky měl také počet použitých permutací. Obecně platilo, že větší množiny permutací vedly k účinnějšímu rozbití symetrií, zároveň však zvyšovaly čas generování klauzulí i velikost výsledné SAT formule. Ukázalo se proto, že při návrhu množiny permutací je nutné hledat kompromis mezi její velikostí a dosaženou účinností.

Dalším důležitým parametrem je hodnota k_{max} , určující hloubku generování **Lex-Leader** podmínek. Experimenty ukázaly, že nejvyšší hodnota k_{max} nemusí vést k nejkratším časům řešení. Přestože větší hodnoty odstraní více symetrií, současně vytvářejí více pomocných proměnných a klauzulí, jejichž zpracování může SAT řešič zpomalit. Optimální hodnota k_{max} se navíc lišila podle typu řešeného problému.

Samostatnou pozornost si zaslouží porovnání s nástrojem **satsuma**. Ukázalo se, že množiny generátorů nalezené tímto nástrojem jsou i při relativně malém počtu permutací velmi kvalitní. V některých případech dosahoval nástroj **satsuma** lepších výsledků než náš přístup, v jiných naopak vedla specializovaná množina permutací navržená přímo pro daný problém k výrazně lepším výsledkům. Důležité je zmínit, že při použití druhé varianty nástroje **satsuma** byly výsledky velmi rozdílné. Varianta **fix** oproti variantě **lex** dosahovala u některých problémů dramaticky lepších výsledků. Pro stejnou množinu permutací generuje klauzule složitějším způsobem, díky kterému rozbíjí symetrie účinněji. Například u problému N královen nebo problému nalezení matice s řádky a sloupci se zadanými součty byly ale výsledky srovnatelné.

Celkově lze konstatovat, že navržený nástroj představuje univerzální přístup k rozbíjení symetrií SAT formulí založený na volitelných množinách permutací. Experimentální výsledky potvrzují, že vhodná volba množiny permutací i parametru k_{max} může mít zásadní vliv na výslednou efektivitu. Současně experimenty odhalily několik zajímavých vlastností, zejména vliv generujících množin permutací a kvalitu množin nalezených nástrojem **satsuma**, které si zaslouží podrobnější teoretické zkoumání v budoucí práci.

Závěr a možnosti dalšího výzkumu

Cílem této diplomové práce bylo navrhnout a implementovat nástroj pro generování klauzulí pro rozbíjení symetrií SAT formulí založený na předdefinovaných množinách permutací a experimentálně ověřit jeho vlastnosti na několika různých typech problémů. Tento cíl se podařilo splnit. V rámci práce byl vytvořen funkční nástroj umožňující využití různých množin permutací, nastavení hloubky generování **Lex-Leader** podmínek i podporu více typů symetrií včetně vícerozměrných problémů. Závěry získané z experimentální části jsou již uvedeny v sekci 3.6.

Teoretická část práce shrnuje základní pojmy z oblasti řešení splnitelnosti, grup, symetrií SAT formulí a metod jejich rozbíjení. Zvláštní pozornost je věnována **Lex-Leader** metodě, která tvoří základ navrženého nástroje. Při návrhu generovaných klauzulí bylo vycházeno zejména z práce [8], ze které byla převzata konstrukce vzorů. Během tvorby práce se ukázalo, že možnosti dalšího rozvoje navrženého přístupu jsou velmi široké. Již práce [8] obsahuje další struktury, pro které bychom mohli hledat zobecnění použitelné i v rámci našeho nástroje. Současně se během návrhu nástroje i experimentální části objevila řada dalších možných rozšíření a proto lze práci chápat především jako základ univerzálního nástroje s potenciálem pro další budoucí rozvoj. Některé z možných směrů rozvoje jsou uvedeny v následujících odstavcích.

Prvním možným směrem je optimalizace samotné implementace. Současná verze nástroje byla navržena především s důrazem na funkčnost a ověření základních principů, nikoli na maximální efektivitu. Implementace je navíc vytvořena v jazyce Python, což může být u výpočetně náročnějších částí limitující. Při větší časové investici do optimalizace, případně při implementaci kritických částí v rychlejší jazyce, například v C++, by bylo možné očekávat výrazné zrychlení nástroje. Rychlost generování klauzulí nebyla pro experimenty provedené v této práci zásadním omezením, přesto by bylo zajímavé dále zkoumat, jak velký praktický dopad by optimalizace implementace mohla mít.

Dalším přirozeným rozšířením je uvažování dalších množin permutací. V současné verzi nástroje je k dispozici pouze omezený výběr množin permutací, které reprezentují určité specifické vlastnosti, například to, zda daná množina generuje celou uvažovanou grupu, nebo zda obsahuje menší či větší počet permutací. Takových množin lze však navrhnout podstatně více. Další výzkum by se proto mohl zaměřit na detailnější návrh a porovnání různých typů množin permutací. Zvláště vhodné by bylo rozšířit experimenty o další množiny permutací pro problém N královen a pro problém nalezení matice s řádky a sloupci se zadanými součty. Zajímavé by také bylo doplnit další dvojice generujících a negenerujících množin, u kterých by bylo možné lépe porovnat vliv vlastnosti generování celé grupy na účinnost rozbíjení symetrií. Množiny použité v této práci, tedy **Neighbors**, **Randtrans** a **Nongen**, byly zvoleny především proto, že jsou vhodné z hlediska velikosti, srozumitelnosti, vzájemné porovnatelnosti a i jednoduchosti implementace.

Významným parametrem pro nástroj je také hodnota $k_{\max}$, která určuje hloubku generování klauzulí. Výsledky experimentální části naznačují, že vhodná volba tohoto parametru může záviset na konkrétním problému i na zvolené množině permutací. Pro některé instance může být výhodné generovat klauzule pouze do

menší části maximální hloubky, zatímco u jiných instancí může vyšší hodnota $k_{\max}$ přinést výraznější redukci počtu symetrických řešení. Dalším směrem výzkumu by proto mohl být návrh metody, která by vhodnou hodnotu $k_{\max}$ odhadovala již ze zadané formule. Jednou z možností by bylo využití metod strojového učení, například modelu trénovaného na větším množství instancí a experimentálních výsledků. Takový model by mohl na základě vlastností vstupní formule, typu symetrie a zvolené množiny permutací doporučit hodnotu $k_{\max}$, která představuje vhodný kompromis mezi silou rozbití symetrie a časem potřebným pro generování klauzulí.

Dále bychom mohli uvažovat rozšíření nástroje o další typy problémů. Ačkoli je nástroj navržen obecně, v současné podobě předpokládá, že je znám typ symetrie dané instance a že odpovídající způsob permutování proměnných je v nástroji implementován. Přidáním dalších typů symetrií by se proto zvýšila použitelnost nástroje na širší třídu problémů. Alternativně by bylo možné nástroj upravit tak, aby společně se SAT formulí přijímal přímo zvolenou množinu permutací na proměnných. Tyto permutace by mohly být přímo použitelné na vektor všech proměnných a nebylo by tedy nutné využívat reprezentaci binární maticí. Tato úprava by výrazně zvýšila obecnost nástroje, zároveň by však přesunula odpovědnost za nalezení nebo vygenerování vhodných množin permutací na jiný nástroj nebo na uživatele.

Literatura

1. BIERE, Armin; HEULE, Marijn; MAAREN, Hans van (ed.). *Handbook of Satisfiability*. Sv. 336. IOS Press, 2021. Frontiers in Artificial Intelligence and Applications. ISBN 978-1-64368-160-3. Dostupné z DOI: 10.3233/FAIA336.
2. GENT, Ian P.; PETRIE, Karen E.; PUGET, Jean-François. Symmetry in Constraint Programming. In: ROSSI, Francesca; BEEK, Peter van; WALSH, Toby (ed.). *Handbook of Constraint Programming*. Elsevier, 2006, sv. 2, s. 329–376. Foundations of Artificial Intelligence. Dostupné z DOI: 10.1016/S1574-6526(06)80014-3.
3. SILVA, João; LYNCE, Inês; MALIK, Sharad. Conflict-Driven Clause Learning SAT Solvers. *Frontiers in Artificial Intelligence and Applications*. 2009, roč. 185. Dostupné z DOI: 10.3233/978-1-58603-929-5-131.
4. ROSSI, Francesca. *Handbook of constraint programming*. Ed. PETER, Van Beek; ROSSI, Francesca; VAN BEEK, Peter; WALSH, Toby. Elsevier Science & Technology, 2014. Foundations of Artificial Intelligence.
5. ANDERS, Markus; BRENNER, Sofia; RATTAN, Gaurav. satsuma: Structure-based Symmetry Breaking in SAT. *CoRR*. 2024, roč. abs/2406.13557. Dostupné z DOI: 10.48550/ARXIV.2406.13557.
6. DEVRIENDT, Jo; BOGAERTS, Bart. *BreakID: Static Symmetry Breaking for ASP (System Description)*. 2016. Dostupné z arXiv: 1608.08447 [cs.AI].
7. FLENER, Pierre; FRISCH, Alan; HNIC, Brahim; KIZILTAN, Zeynep; MIGUEL, Ian; PEARSON, Justin; WALSH, Toby. Breaking Row and Column Symmetries in Matrix Models. In: 2002. ISBN 978-3-540-44120-5. Dostupné z DOI: 10.1007/3-540-46135-3_31.
8. CODISH, Michael; JANOTA, Mikoláš. *Breaking Symmetries from a Set-Covering Perspective*. 2025. Dostupné z arXiv: 2502.10056 [cs.LO].
9. HAKEN, Armin. The Intractability of Resolution. *Theor. Comput. Sci.* 1985, roč. 39, s. 297–308.
10. ZHOU, Neng-Fa. *Yet Another Comparison of SAT Encodings for the At-Most-K Constraint*. 2020. Dostupné z arXiv: 2005.06274 [cs.LO].
11. OEIS FOUNDATION INC. (2026). *The On-Line Encyclopedia of Integer Sequences*. 2026. Dostupné také z: <https://oeis.org>.
12. DIMACS CHALLENGE ORGANIZATION. *Satisfiability: Suggested Format*. Rutgers University, 1993. Tech. zpr. Center for Discrete Mathematics a Theoretical Computer Science. Archived at <http://dimacs.rutgers.edu/archive/Challenges/>.
13. IGNATIEV, Alexey; MORGADO, Antonio; MARQUES-SILVA, Joao. PySAT: A Python Toolkit for Prototyping with SAT Oracles. In: *SAT*. 2018, s. 428–437. Dostupné z DOI: 10.1007/978-3-319-94144-8_26.
14. TSEITIN, G. S. On the complexity of derivations in the propositional calculus. *Studies in Constructive Mathematics and Mathematical Logic*. 1968.

15. BIERE, Armin; FALLER, Tobias; FAZEKAS, Katalin; FLEURY, Mathias; FROLEYKS, Nils; POLLITT, Florian. CaDiCaL, Gimsatul, IsaSAT and Kissat Entering the SAT Competition 2024. In: HEULE, Marijn; ISER, Markus; JÄRVISALO, Matti; SUDA, Martin (ed.). *Proc. of SAT Competition 2024 – Solver, Benchmark and Proof Checker Descriptions*. University of Helsinki, 2024, sv. B-2024-1, s. 8–10. Department of Computer Science Report Series B. Dostupné také z: <http://hdl.handle.net/10138/584822>.
16. LAGNIEZ, Jean-Marie; MARQUIS, Pierre. An Improved Decision-DNNF Compiler. In: *Proceedings of the Twenty-Sixth International Joint Conference on Artificial Intelligence, IJCAI-17*. 2017, s. 667–673. Dostupné z DOI: 10.24963/ijcai.2017/93.
17. HUNTER, J. D. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*. 2007, roč. 9, č. 3, s. 90–95. Dostupné z DOI: 10.1109/MCSE.2007.55.
18. TEAM, The pandas development. *pandas-dev/pandas: Pandas*. Zenodo, 2020. Latest. Dostupné z DOI: 10.5281/zenodo.3509134.
19. HARRIS, Charles R.; MILLMAN, K. Jarrod; WALT, Stéfan J. van der; GOMMERS, Ralf; VIRTANEN, Pauli; COURNAPEAU, David; WIESER, Eric; TAYLOR, Julian; BERG, Sebastian; SMITH, Nathaniel J.; KERN, Robert; PICUS, Matti; HOYER, Stephan; KERKWIJK, Marten H. van; BRETT, Matthew; HALDANE, Allan; RÍO, Jaime Fernández del; WIEBE, Mark; PETERSON, Pearu; GÉRARD-MARCHANT, Pierre; SHEPPARD, Kevin; REDDY, Tyler; WECKESSER, Warren; ABBASI, Hameer; GOHLKE, Christoph; OLIPHANT, Travis E. Array programming with NumPy. *Nature*. 2020, roč. 585, č. 7825, s. 357–362. Dostupné z DOI: 10.1038/s41586-020-2649-2.

Seznam obrázků

3.1	Čas kissat a čas nástroje pro php_pig21_hol20	30
3.2	Čas kissat a čas nástroje pro php_pig25_hol20	30
3.3	Čas kissat a čas nástroje pro php_pig31_hol30	31
3.4	Čas kissat a čas nástroje pro php_pig35_hol15	31
3.5	Čas kissat a čas nástroje pro php_pig40_hol10	31
3.6	Čas nástroje + čas d4 a počet modelů pro queens_n10	33
3.7	Čas nástroje + čas d4 a počet modelů pro queens_n11	33
3.8	Čas nástroje + čas d4 a počet modelů pro queens_n12	33
3.9	Čas nástroje + čas d4 a počet modelů pro queens_n13	33
3.10	Čas nástroje + čas d4 a počet modelů pro queens_n14	34
3.11	Čas nástroje + čas d4 a počet modelů pro rcsuns_rc10_rcrand2 .	35
3.12	Čas nástroje + čas d4 a počet modelů pro rcsuns_rc12_rcrand2 .	35
3.13	Čas kissat a čas nástroje pro rcsuns_rc14_rcrand13	35
3.14	Čas kissat a čas nástroje pro rcsuns_rc15_rcrand3	36
3.15	Čas nástroje + čas d4 a počet modelů pro golf_g3_p3_w3	37
3.16	Čas nástroje + čas d4 a počet modelů pro latinsq_ls6	38
3.17	Čas nástroje + čas d4 a počet modelů pro latinsq_ls7	38
3.18	Čas kissat a čas nástroje pro latinsq_ls8	38
3.19	Čas nástroje + čas d4 a počet modelů pro golf_g3_p3_w3	39
3.20	Čas kissat a čas nástroje pro golf_g4_p4_w6	39
3.21	Čas kissat a čas nástroje pro golf_g5_p3_w6	40

A Přílohy

A.1 Zdrojový kód a data pro experimentální část

V příloze práce jsou k dispozici hlavní zdrojové kódy v jazyce Python, grafy a tabulky s výsledky experimentální části a všechny výstupy našeho nástroje použité při testování. Součástí přílohy jsou také grafy, které z důvodu úspory místa nebyly zařazeny do hlavního textu práce. Aktuální verze nástroje a případné budoucí výsledky dalších testování budou dostupné na <https://github.com/MartinStruk/symmetries>. Následuje podrobnější popis jednotlivých částí přílohy.

1. **plots:** Všechny vygenerované grafy s výsledky. Význam jednotlivých grafů je popsán v grafech samotných.
2. **problems:** Všechny testované výstupy, jejichž výsledky byly srovnávány v experimentální části.
3. **tables:** Tabulky se souhrny výsledků z experimentální části.
4. **geninput.py:** Zdrojový kód nástroje pro generování vstupních instancí uvažovaných problémů.
5. **sym.py:** Zdrojový kód hlavního nástroje práce, který pro zadanou formuli vygeneruje klauzule pro rozbíjení symetrií.

Spuštění `sym.py` nebo `geninput.py` se provádí z příkazové řádky se specifickými argumenty. Konkrétní pořadí a formát argumentů lze zjistit spuštěním kódu s argumentem `-h`.